Fully dynamic union-volume estimation in the oracle model

Sujoy Bhore, Karl Bringmann, Timothy M. Chan, Yanheng Wang

arxiv:2602.16306; CC BY 4.0

Editorial scope. The construction maintains adaptive Poisson sample digests under deletion and combines decremental estimators through a dynamic rebuilding hierarchy, controlling support ensembles and amortized sample refreshes to obtain polylogarithmic update and query time. Complete native author full-version proofs retained under own CCBY4; no experiment or algorithm implementation rerun.

Allowed standard prerequisites (editorial).

- Poisson concentration, geometric approximation, fixed-dimensional ILP and sparse recovery. Author statements and bibliography retained. All source-specific digest, suffix sampling and implicit geometric hash enumeration proofs included.

1 Introduction

In the union volume estimation problem, we are given objects $X_1, \ldots, X_n$, which are assumed to be measurable sets in $\mathbb{R}^d$, and the task is to $(1 \pm \varepsilon)$-approximate the volume of $\bigcup_{i=1}^{n} X_i$.[1] This is a fundamental problem in computer science, with connections to several areas such as network reliability [Kar01, KLM89], DNF counting [KLM89], general model counting [PVBM23, CMV13],

[1] A discrete variant of the problem, estimating the cardinality of the union of sets $X_1, \ldots, X_n \subset \mathbb{Z}^d$, can be easily reduced to the continuous version, by replacing each grid point with a unit grid cell.

probabilistic databases [KKS08, DS07, RDS06], and probabilistic query evaluation on databases [CZB+20, CTG+23]. Perhaps the most popular special case in computational geometry is Klee's measure problem [Kle77], where each object is a d-dimensional axis-aligned box of the form $[a_1, b_1] \times \cdots \times [a_d, b_d]$. There has been extensive work on Klee's measure problem (either exact or approximate) in constant dimensions [OY91, Cha10, Cha13, Kün22, GK23, BLN+25]. To capture a wide variety of input objects with a unified framework, Karp and Luby [KL85] introduced an oracle model. An algorithm in this model can access each object X_i exclusively via three types of queries: (i) X_i.size() gives the volume of X_i, (ii) X_i.sample() samples a point uniformly at random from the set X_i, and (iii) X_i.contains(x) tests whether a given point x is contained in X_i. The time complexity of the algorithm is measured by the number of queries plus the number of additional steps; that is, each query is assumed to take unit time. In a seminal result, Karp, Luby, and Madras [KLM89] gave an algorithm in the oracle model that solves union volume estimation in time $O(n/\varepsilon^2)$ with constant success probability, where ε is the approximation precision. Black-box usage of this result also yields the best known algorithms for several applications, including DNF counting. (For Klee's measure problem, it implies an $O(nd/\varepsilon^2)$-time approximation algorithm [BF10]; in small dimensions, this was improved to $O((n + 1/\varepsilon^2)\log^{O(d)} n)$ time at last year's SoCG [BLN+25].) Only recently has a matching lower bound been established: $\Omega(n/\varepsilon^2)$ queries are necessary to solve union volume estimation in the oracle model [BLN+25]. With the static setting now fully understood, attention naturally shifts to the dynamic setting, where the input evolves over time through insertions and deletions of objects. The study of dynamic union volume estimation has a twofold motivation. On one hand, it captures modern data-driven applications where datasets are inherently mutable, and volume estimates must be maintained continuously rather than recomputed from scratch after each update. On the other hand, the dynamic setting presents a compelling theoretical challenge, as it is far from obvious whether the tools developed for the static setting can be dynamized efficiently. Indeed, dynamic geometric problems have repeatedly proven to be substantially more difficult than their static counterparts. An exten-

sive body of work in recent years deals with dynamic geometric problems such as set cover [ACS+22, CHSX25], independent set [HNW20, BNTW24, BC25], planar point location [Nek21], connectivity [CPR11, CH24], and nearest neighbor search [AAS18], yet in this broader context dynamic union volume estimation remains largely unexplored. The following question has not been resolved even for very simple classes of objects such as triangles in the plane:

> Q1: Can we solve dynamic union volume estimation in the oracle model under insertions and deletions of objects in poly $\left(\log n, \frac{1}{\varepsilon}\right)$ update and query time?[2]

An even more demanding paradigm is the (one-pass) streaming setting, where the algorithm has to not only handle updates, but use space that is typically required to be polylogarithmic in the input size. Earlier work has explored streaming algorithms under strong restrictions on the input objects. For example, Tirthapura and Woodruff [TW12] designed a dynamic streaming algorithm for Klee's measure problem on axis-aligned boxes. However, the problem remains unaddressed for more general classes of objects:

> Q2: Can we solve dynamic union volume estimation in the oracle model under insertions and deletions of objects in poly $\left(\log n, \frac{1}{\varepsilon}\right)$ space as well as update and query time?

In the restricted insertion-only setting, Meel, Vinodchandran, and Chakraborty [MVC21] gave a positive answer to this question, using $O\left(\frac{1}{\varepsilon^2}\log^2(n)(\log\log(n)+\log(\frac{1}{\varepsilon}))\right)$ time per update and query, and $O\left(\frac{1}{\varepsilon^2}\log(n)\right)$ space in the oracle model.[3] However, their algo-

[2] Low-dimensional fat objects can be easily handled using quadtrees; see Section A for details. The problem becomes much more challenging for non-fat objects.

[3] In this paper, we assume that points from the input objects and $O(\log n)$-bit integers fit into a machine word, and space complexity counts the number of machine words. An arithmetic operation on machine words is assumed to take unit time. Both are natural assumptions for a continuous universe. Some previous work such as [MVC21] focuses on sets from a finite universe and studies bit complexity; when stating previous work in the introduction, we have translated their results into our setting. In the finite universe Ω, different trade-offs between $\log(n)$ and $\log|\Omega|$ were obtained in [MCV22, NVG+24].

rithm cannot handle deletions.

2 Our Contributions

2.1 Dynamic algorithm with insertions and deletions

We answer our driving question (Q1) positively, not only for 2D triangles, 3D simplices, and other common types of objects encountered in computational geometry, but in the full generality of the oracle model. Our new dynamic algorithm can handle arbitrary sequences of both insertions and deletions. More precisely, we consider an initially empty multiset of objects $\mathcal{X}$ under the updates insert(X), which adds object X to $\mathcal{X}$, and delete(X), which removes object X from $\mathcal{X}$. Upon query estimate(), the algorithm outputs a $(1\pm\varepsilon)$-approximation to the volume vol $\left(\bigcup_{X\in\mathcal{X}} X\right)$. During insertion/deletion the algorithm is given oracle access to the inserted/deleted object. We also need the ability to store objects (or their oracles), in case of future lookup of the previously inserted objects. We make the natural assumption that (i) the argument of insert(X) and delete(X) is a pointer to the oracle of X, (ii) we can store a pointer in a machine word, and (iii) knowing the pointer allows us to access the oracle to which the pointer is pointing. In this interpretation, $\mathcal{X}$ is treated as a multiset of pointers: the same pointer can be inserted multiple times, and upon deletion, the multiplicity of the pointer is decreased by one. Naturally, the multiplicity of any pointer must be non-negative throughout the process (in particular, objects being deleted must have previously been inserted).

Theorem 2.1 (Section 5)**.** There is a dynamic algorithm in the oracle model that maintains a multiset $\mathcal{X}$ of objects under updates insert(X) and delete(X). Upon query estimate(), it outputs a $(1\pm\varepsilon)$-approximation to vol $\left(\bigcup_{X\in\mathcal{X}} X\right)$ with high probability. The amortized expected update time is $O(\log^5(n/\varepsilon)/\varepsilon^2)$, the query time is $O(\log^4(n)/\varepsilon^2)$, and the space complexity is $O(n+\log^4(n)/\varepsilon^2)$.

Note that since the algorithm operates in the general oracle model, it works well even for high-dimensional objects (e.g., for simplices in $\mathbb{R}^d$, with polynomial dependence on d).

2.2 Streaming algorithm with insertions and suffix queries

In our remaining results, we turn to streaming algorithms and our driving question (Q2) and show that polylogarithmic time and space can be achieved in certain settings. We first consider the sliding window setting [DGIM02], which supports insertions of objects and querying over the last w objects for a fixed window size w; thus, objects are deleted in the same order as they are inserted. Numerous results and techniques have been developed in the sliding window setting (see [Mut05, BBD$^+$02, Bra16] for an overview and [WZ21, JWZ22, FSW25] for recent work), but no such result was known for union volume estimation. We obtain the first efficient sliding-window algorithm for union volume estimation. Again, it works in the general oracle model! Moreover, the result holds in an extension of the sliding-window setting, where at each time $t = 1, \ldots, n$ an object X_t is inserted (but cannot be deleted), and a query estimate(s) may return a $(1 \pm \varepsilon)$-approximation to the volume of the suffix $\text{vol}(X_s \cup \cdots \cup X_t)$ with high probability for any number s the user specifies.

Theorem 2.2 (Section 6)**.** There is an algorithm that implicitly maintains a sequence of objects $X_1, \ldots, X_t$ of aspect ratio M. The algorithm supports update insert(X), which appends an object X (as X_{t+1}) to the sequence, and query estimate(s), which outputs a $(1 \pm \varepsilon)$-approximation to $\text{vol}(X_s \cup \cdots \cup X_t)$ with high probability. The update time is $O(\log^2(n/\varepsilon)/\varepsilon^2)$, the query time is $O(\log(n)/\varepsilon^2)$, and the space complexity is $O(\log(M)\log(n)/\varepsilon^2)$.

Our algorithm not only generalizes previous insertion-only results [MVC21], but is also remarkably simple. Note that the space bound depends on the aspect ratio M, defined as the ratio of the largest volume of any object ever inserted and the smallest volume of any object ever inserted. In many applications, M is a polynomial in n and thus $\log(M) = O(\log n)$. Furthermore, geometric algorithms in the sliding window setting [FKZ05, CS06] usually have logarithmic dependencies on parameters such as the aspect ratio or spread.

2.3 Streaming algorithm for convex bodies with insertions and deletions

Lastly, we consider streaming algorithms that support both insertions and deletions. We manage to develop an efficient algorithm for a reasonably wide class of geometric objects—namely, convex bodies in any constant dimension:

Theorem 2.3 (Section 7)**.** Given $R \geq r > 0$, there is an algorithm that implicitly maintains a multiset $\mathcal{X}$ of convex bodies in $[0, R]^d$, where each convex body contains a ball of radius r, under updates insert(X) and delete(X). Upon query estimate(), it outputs a $(1 \pm \varepsilon)$-approximation to vol $\left(\bigcup_{X \in \mathcal{X}} X\right)$ with high probability. The expected update and query time and space are $O(\text{polylog}(\frac{nR}{\varepsilon r})/\varepsilon^2)$ for constant d.

The bounds depend polylogarithmically on R/r, which is common in volume estimation algorithms for a single convex object [DFK91] as well as geometric streaming algorithms [Ind04, FIS05]. Previously, Tirthapura and Woodruff obtained an efficient dynamic streaming algorithm only for axis-aligned boxes [TW12] (with poly$(\log n, \log \Delta, \frac{1}{\varepsilon}, d)$ update time and space in a discrete universe $\{1, \ldots, \Delta\}^d$). To emphasize the generality of our algorithm, note that many non-convex objects can be decomposed into a small number of convex objects, e.g., non-convex polyhedra of constant combinatorial complexity in constant dimensions can be decomposed into a constant number of simplices, to which our algorithm can be applied.

3 Technical Overview

3.1 Dynamic algorithm with insertions and deletions

Karp, Luby and Madras' algorithm [KLM89] and subsequent work are based on the observation that we can estimate the volume of a union U by maintaining a finite random subset $S^{(\ell)} \subset U$ that selects every point independently with probability density $1/2^\ell$. It turns out that the "right" choice of ℓ is one such that $|S^{(\ell)}| \approx \log(n)/\varepsilon^2$; in this case $2^\ell |S^{(\ell)}|$ provides a good estimate of the volume of U. Meel, Vinodchandran, and Chakraborty [MVC21] described

a particularly simple way to maintain such a random set $S^{(\ell)}$ for insertion-only updates: when a new object X is inserted, we remove from $S^{(\ell)}$ all points that are contained in X, then add to $S^{(\ell)}$ freshly sampled points in X with density $1/2^\ell$. This step is inexpensive for the right choice of ℓ since $|S^{(\ell)}|$ is logarithmic. If $|S^{(\ell)}|$ becomes too big, we increment ℓ (thereby halving the sampling rate and also $|S^{(\ell)}|$ in expectation) and move on. Difficulty arises in the presence of deletions: the volume is no longer monotonically increasing, and the right choice of ℓ may increase or decrease over time. Our first observation is that the deletion-only case can be handled by a variant of Meel, Vinodchandran, and Chakraborty's insertion-only algorithm. Roughly, we can maintain $S^{(\ell)}$ for the right ℓ by keeping a counter on the number of objects containing each sample point (recall that $|S^{(\ell)}|$ is logarithmic for the right ℓ). When an object X is deleted, we decrement the counters of all points in $S^{(\ell)} \cap X$, and when a counter drops to 0, the point is removed from $S^{(\ell)}$. When the number of surviving sample points drops below logarithmic, we decrease ℓ, rebuild $S^{(\ell)}$ by iterating over all objects that has not been deleted (recall that can store pointers to all objects), and reinitialize the counters. A popular technique in dynamic computational geometry by Bentley and Saxe [BS80], called the logarithmic method, transforms deletion-only data structures into fully dynamic ones. It works by decomposing a fully dynamic set into logarithmically many deletion-only subsets, increasing the update and query time by typically just a logarithmic factor. The logarithmic method applies to so-called decomposable problems that satisfy the following property: if the input is decomposed into subsets, then the answer to the original problem can be quickly obtained from the answers to the individual subsets. Unfortunately, union volume estimation is not a decomposable problem: We cannot determine the volume of the union of $\mathcal{X}_0 \cup \cdots \cup \mathcal{X}_{\log n}$ just from the volume of the union of each $\mathcal{X}_i$. Nevertheless, we show that a novel variant of the logarithmic method can still work. Recall that the deletion-only data structure already maintains a sample for each $\mathcal{X}_i$. For each sample point we keep track of more counters—specifically, how many sets from the other $\mathcal{X}_i$'s contain the point. These counters provide enough information to avoid double counting and help us estimate the union volume accurately. One issue though is that many counters need to be adjusted when one of the deletion-

only data structures changes ℓ and recomputes $S^{(\ell)}$. Fortunately, we manage to bound the total number of such changes, so the amortized cost for maintaining all counters remains polylogarithmic. See Section 5 for details.

3.2 Streaming algorithm with insertions and suffix queries

The dynamic algorithm with arbitrary insertions and deletions needs linear space because the data structure rebuilds on the remaining objects when the "right" ℓ decreases. However, in the sliding window setting and more generally the setting of suffix queries, the remaining objects are contiguous in time and need not be stored explicitly. We obtain a streaming algorithm in this case by a simple yet non-trivial generalization of Meel, Vinodchandran, and Chakraborty's algorithm. Specifically, we keep the timestamp when each point in $S^{(\ell)}$ is sampled. For each ℓ we keep track of the largest timestamp $s^{(\ell)}$ such that discarding points older than $s^{(\ell)}$ from $S^{(\ell)}$ makes $|S^{(\ell)}|$ logarithmically bounded. This timestamp $s^{(\ell)}$ is monotonically increasing over time for each fixed ℓ. To estimate the volume of the union of $X_s \cup \cdots \cup X_t$ for a given s at time t, we identify the smallest ℓ for which $s^{(\ell)} \leq s$, and output $2^\ell |S^{(\ell)} \cap (X_s \cup \ldots \cup X_t)|$. See Section 6 for details.

3.3 Streaming algorithm for convex bodies with insertions and deletions

Our streaming algorithm for low-dimensional convex objects for arbitrary insertions and deletions is based on another simple sampling strategy. First, it is not difficult to discretize the space $\mathbb{R}^d$ to a grid $[\Delta]^d$ for some large Δ, so that the volume of the union U is close to the number of grid points in $U \cap [\Delta]^d$. The idea now is to select a random subset $R^{(\ell)}$ of the grid $[\Delta]^d$ in the beginning, and maintain the intersection $S^{(\ell)} := U \cap R^{(\ell)}$ during the updates. Using a pairwise independent family of hash functions, $R^{(\ell)}$ can be represented implicitly with logarithmically many bits. When we insert or delete an object X, we just need to insert or delete $X \cap R^{(\ell)}$ to or from $S^{(\ell)}$ (viewed as a multiset of points). An issue is that the space usage may be large for a $S^{(\ell)}$ with a "wrong" sampling rate, and we do not know which ℓ is the right choice beforehand.

Furthermore, a wrong choice may become right at a later time. This issue can be resolved by representing $S^{(\ell)}$ by data structures for sparse vectors [FIS05, JST11], which support adding numbers to a given coordinate, as well as recovering the vector when its norm is below a threshold. In this way, we essentially just pay for the space at the right sampling rate. A question remains: how to enumerate the points of $X \cap R^{(\ell)}$ for a given object X and for our implicitly represented set $R^{(\ell)}$? For the case where the objects are rotated (non-axis-aligned) boxes in constant dimension, and for a particular pairwise independent family of hash functions, we observe that this subproblem directly reduces to integer linear programming (ILP) with a constant number of constraints and variables, and therefore can be solved efficiently by well-known algorithms [Len83]. For more general convex objects in constant dimension, we bound each object X by a small rotated box $B \supseteq X$, use an ILP to enumerate the points of $B \cap R^{(\ell)}$ and discard those not in X. By "small" we mean that the volume of B is comparable to that of X, so in particular the number of discarded points is small. See Section 7 for the details. Our approach is related to Tirthapura and Woodruff's algorithm [TW12] for axis-aligned boxes. Their idea was to directly adapt a streaming algorithm for approximately counting distinct elements to estimate $|U \cap [\Delta]^d|$: to insert or delete an object X, we cannot afford to literally insert or delete all points in $|X \cap [\Delta]^d|$, but they showed how to efficiently simulate the effect for a specific distinct elements algorithm in the case of axis-aligned boxes. Our approach is more modular in that we apply sparse recovery only as a black box. Our main contribution however is the realization, via solving an ILP, that this type of approach works far more generally than for axis-aligned boxes, namely, for general convex objects.

4 Preliminaries

4.1 Technical setup

We assume that all relevant parameters (the precision ε, an upper bound n on the number of operations, etc.) are given to the algorithms at initialization. Knowing n is in line with the usual assumptions in streaming algorithms; it can easily be avoided

but this complicates notation. The phrase "with high probability" means that the probability is at least $1 - n^{-\Omega(1)}$. We denote $[n] := \{1, 2 \ldots, n\}$. The notation $a \in b \pm c$ means that $b - c \le a \le b + c$. Similarly, $a \in (1 \pm \varepsilon)b$ means that $(1 - \varepsilon)b \le a \le (1 + \varepsilon)b$, and we say that a is a $(1 \pm \varepsilon)$-approximation to b.

4.2 Reduction of volume range

The following lemma reduces the volumes of objects to a narrow range $[(3n/\varepsilon)^2, (3n/\varepsilon)^4]$, without affecting efficiency much.

Lemma 4.1. Suppose that $(1 \pm \varepsilon)$ union volume estimation under insertions and deletions (resp. suffix queries) can be solved in $S(n, \varepsilon)$ space and $T(n, \varepsilon)$ update time when objects have volumes in $[(3n/\varepsilon)^2, (3n/\varepsilon)^4]$. Then the problem without volume restriction can be solved in $S(n, \varepsilon/3) \cdot O\left(\frac{\log(M)}{\log(n/\varepsilon)} + 1\right)$ space and $T(n, \varepsilon/3) + O(\log n)$ update time. Moreover, if $S(n, \varepsilon)/n$ is non-decreasing in n, then the space bound can be strengthened to $O(S(n, \varepsilon/3))$.

Proof. Write $m := 3n/\varepsilon$. Let $\mathcal{A}$ be a union volume estimation algorithm using $S(n, \varepsilon)$ space and $T(n, \varepsilon)$ update time when objects have volumes in $[(3n/\varepsilon)^2, (3n/\varepsilon)^4]$. We construct another algorithm that handles general volumes. Let class $\mathcal{C}_\ell$ collect all input objects whose volume is in $(m^{\ell-1}, m^{\ell+1}]$. Non-empty classes are said to be active. Note that each object appears in exactly two classes, and there are $O(\frac{\log M}{\log m} + 1)$ active classes. We organize the active classes in a binary search tree ordered by indices ℓ. However, we do not store $\mathcal{C}_\ell$ explicitly but only a counter $|\mathcal{C}_\ell|$. Moreover, we run a copy $\mathcal{A}_\ell$ of $\mathcal{A}$ with parameter $\varepsilon/3$ on each active class $\mathcal{C}_\ell$, after scaling the objects in it by a factor of $m^{3-\ell}$. The objects inside the class now have volumes in $(m^2, m^4]$, so algorithm $\mathcal{A}$ can solve union volume estimation in space $S(n, \varepsilon/3)$ and update time $T(n, \varepsilon/3)$. When an object X is inserted, we determine from X.size the two classes $\mathcal{C}_\ell, \mathcal{C}_{\ell'}$ containing X. If C_ℓ or $C_{\ell'}$ was empty before the insertion, then we add it to the binary search tree. We increment the counters $|\mathcal{C}_\ell|$ and $|\mathcal{C}_{\ell'}|$. Finally, we insert X (after scaling) into $\mathcal{A}_\ell$ and $\mathcal{A}_{\ell'}$. Deletion is handled similarly, and if some counter drops to zero we remove the class from the tree. To estimate the volume of the union, we locate the second largest ℓ in the tree, and output the estimate of $\mathcal{A}_\ell$ scaled by a factor of $m^{\ell-3}$. It is clear

that the space and update time of this new algorithm is as stated. To see correctness, we first observe that the highest class is a subset of the second highest. Indeed, every object X is in exactly two classes; if it is in the highest then it needs to be in the second highest, too. Next we write z for the correct output, z_ℓ for the volume of union of the second highest class, and $z_{<\ell}$ for the volume of union of lower classes. Note that there exists an object of volume at least m^ℓ, because ℓ is not the highest class. In particular, $z_\ell \geq m^\ell$. On the other hand, every object contained in some lower class but not in the second highest class has volume at most $m^{\ell-1}$ by definition, so $z_{<\ell} \leq n \cdot m^{\ell-1} \leq \varepsilon z_\ell/3$. Therefore,

$$z_\ell \leq z \leq z_\ell + z_{<\ell} \leq \left(1 + \frac{\varepsilon}{3}\right) z_\ell.$$

It follows that any $(1 \pm \frac{\varepsilon}{3})$-approximation to z_ℓ is a $(1 \pm \frac{\varepsilon}{3})^2 \subseteq (1 \pm \varepsilon)$-approximation to z. For the "moreover" part, let us write $n_\ell := |\mathcal{C}_\ell|$. The space of the same algorithm can be bounded up to constant factors by

$$\sum_\ell S\left(n_\ell, \frac{\varepsilon}{3}\right) = \sum_\ell n_\ell \cdot \frac{S(n_\ell, \frac{\varepsilon}{3})}{n_\ell} \leq \sum_\ell n_\ell \cdot \frac{S(n, \frac{\varepsilon}{3})}{n} \leq 2n \cdot \frac{S(n, \frac{\varepsilon}{3})}{n} = 2S\left(n, \frac{\varepsilon}{3}\right),$$

where the second step uses the assumption that $S(n, \varepsilon)/n$ is non-decreasing, and the third step uses the fact that each object appears in two classes. □

4.3 Volume estimation via sampling

As in all previous work, our algorithms exploit the close relation between volume estimation and sampling. The notion of ℓ-samples is central:

Definition 4.2 (ℓ-sample)**.** Let U be a measurable set and $\ell \in \mathbb{Z}$. A random finite subset $S \subset U$ is an ℓ-sample of U, denoted $S \sim \text{Pois}(U, \ell)$, if $|S \cap V| \sim \text{Pois}(\text{vol}(V)/2^\ell)$ for every measurable subset $V \subseteq U$.

It follows from this definition that: (i) if $S \sim \text{Pois}(U, \ell)$, then $S \cap V \sim \text{Pois}(V, \ell)$ for any measurable subset $V \subseteq U$; (ii) if $S \sim \text{Pois}(U, \ell)$ and $T \sim \text{Pois}(V, \ell)$ are independent and U, V are disjoint,

then $S \cup T \sim \text{Pois}(U \cup V, \ell)$. Note that an ℓ-sample S of U has expected size $\text{vol}(U)/2^\ell$. If ℓ is small, then intuitively $2^\ell|S|$ should concentrate around $\text{vol}(U)$. The threshold of being small is given in Definition 4.3, and its justification in Lemma 4.4.

Definition 4.3 (level)**.** For a measurable set U, we define $\text{level}(U)$ as the unique integer such that $\frac{32\log n}{\varepsilon^2} \le \frac{\text{vol}(U)}{2^{\text{level}(U)}} < \frac{64\log n}{\varepsilon^2}$.

Lemma 4.4. If S is an ℓ-sample of U for $\ell \le \text{level}(U)$, then $2^\ell|S| \in (1 \pm \varepsilon)\,\text{vol}(U)$ with probability at least $1 - O(n^{-8})$.

Proof. Note that $|S| \sim \text{Pois}(\text{vol}(U)/2^\ell)$. By concentration of Poisson variables [Can19],

$$\mathbb{P}\left(\left|2^\ell|S| - \text{vol}(U)\right| \ge \varepsilon\,\text{vol}(U)\right) \le 2\exp\left(\frac{-\varepsilon^2\,\text{vol}(U)}{4\cdot 2^\ell}\right) \le 2\mathrm{e}^{-8\log n} = O(n^{-8}).$$

Let U denote the union we are interested in. Each of our algorithms morally maintains an ℓ-sample $S^{(\ell)}$ of U for every $\ell \in \mathbb{Z}$ throughout the updates; upon query, it tries to identify some $L \le \text{level}(U)$ and outputs $2^L|S^{(L)}|$. In the analysis, we morally apply Lemma 4.4 to conclude that $2^L|S^{(L)}|$ is a good estimate of $\text{vol}(U)$. However, there are two pitfalls. The first pitfall is that the identification of L actually depends on the outcomes of the random sets $S^{(\ell)}$, so L is a random variable. Conditioned on L, it is not true that $S^{(L)} \sim \text{Pois}(U, L)$, and Lemma 4.4 does not apply directly. The second pitfall is that we cannot afford to maintain all sets $S^{(\ell)}$ at once. Our algorithms only explicitly store those $S^{(\ell)}$ with $\ell \approx \text{level}(U)$; since such $S^{(\ell)}$ has an expected size roughly $\text{vol}(U)/2^{\text{level}(U)} = \Theta(\log(n)/\varepsilon^2)$ it can be stored in low space. In contrast, for $\ell \ll \text{level}(U)$ then $S^{(\ell)}$ is too large to sample or store, so these sets must be compressed or discarded. Naturally, this further complicates the algorithms. We use different techniques in different settings to avoid the aforementioned pitfalls. □

5 Dynamic Algorithm With Insertions and Deletions

In this section, we prove Theorem 2.1 by presenting a fully dynamic (non-streaming) algorithm for union volume estimation in linear space and polylogarithmic update time. Due to Lemma 4.1,

we assume without loss of generality that the volumes of input objects are in the range $[(3n/\varepsilon)^2, (3n/\varepsilon)^4]$.

5.1 Preparations

We will use the Karp–Luby–Madras algorithm as a subroutine:

Theorem 5.1 (Karp, Luby, Madras [KLM89])**.** Let $n \in \mathbb{N}$ be a parameter. There is an algorithm KLM($\mathcal{X}$) that computes a (1 ± 0.5)-approximation to $\text{vol}(\bigcup_{X \in \mathcal{X}} X)$ with probability at least $1 - n^{-3}$ and runs in time $O(|\mathcal{X}| \log n)$.

We need a somewhat technical notion that is "close" to an ℓ-sample at the right level ℓ.

Definition 5.2. Let U be a measurable set. A family of pairs $\{(\ell_1, R_1), \ldots, (\ell_N, R_N)\}$ is called an ensemble of U if $\ell_k \in \text{level}(U) \pm 1$ and $R_k \sim \text{Pois}(U, \ell_k)$ for all $k \in [N]$. A random pair (L, S) is called a (δ, N)-digest of U if there exists an ensemble $\mathcal{E}$ of U such that $|\mathcal{E}| \le N$ and $\mathbb{P}[(L, S) \notin \mathcal{E}] \le \delta$. The ensemble $\mathcal{E}$ is called a support ensemble of the digest (L, S).

Lemma 5.3. Fix measurable sets $V \subseteq U$. If $\mathcal{E}$ is an ensemble of U, then with probability at least $1 - \frac{2|\mathcal{E}|}{n^4}$, we have $\left|2^\ell |R \cap V| - \text{vol}(V)\right| \le \varepsilon \,\text{vol}(U)$ for all $(\ell, R) \in \mathcal{E}$.

Proof. Consider an arbitrary $(\ell, R) \in \mathcal{E}$. By definition, $\ell \le \text{level}(U) + 1$ and $R \sim \text{Pois}(U, \ell)$. In particular, $|R \cap V| \sim \text{Pois}(\lambda)$ where $\lambda := \text{vol}(V)/2^\ell$. Let us write $d := \varepsilon \,\text{vol}(U)/2^\ell \ge \varepsilon\lambda$. By concentration of Poisson variables [Can19],

$$\mathbb{P}\left(\left|2^\ell |R \cap V| - \text{vol}(V)\right| > \varepsilon \,\text{vol}(U)\right) \;=\; \mathbb{P}(||R \cap V| - \lambda| > d)$$
$$\le 2\exp\left(-\frac{d^2}{2(\lambda + d)}\right) \le 2\exp\left(-\frac{\varepsilon d}{2(1+\varepsilon)}\right) = 2\exp\left(-\frac{\varepsilon^2 \,\text{vol}(U)}{2(1+\varepsilon)\cdot 2^\ell}\right) \le 2\mathrm{e}^{-4\log n} \le \frac{2}{n^4}$$

where the second last step uses $\frac{\text{vol}(U)}{2^\ell} \ge \frac{\text{vol}(U)}{2^{\text{level}(U)+1}} \ge \frac{16 \log n}{\varepsilon^2}$. Taking a union bound over all $(\ell, R) \in \mathcal{E}$ proves the lemma. □

Corollary 5.4. Fix measurable sets $V \subseteq U$. If S is a (δ, N)-digest of U, then

$$\mathbb{P}\left(\left|2^L |S \cap V| - \text{vol}(V)\right| \le \varepsilon \,\text{vol}(U)\right) \ge 1 - \delta - \frac{2N}{n^4}.$$

Proof. Let $\mathcal{E}$ be a support ensemble of (L, S), so $|\mathcal{E}| \leq N$ and $\mathbb{P}[(L, S) \notin \mathcal{E}] \leq \delta$. The event in the statement is implied by $(L, S) \in \mathcal{E}$ and the event in Lemma 5.3. □

Lemma 5.5. Let $n \in \mathbb{N}$ and $\varepsilon \in (0, \frac{1}{4}]$ be parameters. There is an algorithm erase-resample($\mathcal{X}, L$) that, given a multiset $\mathcal{X}$ of $m \leq n$ objects and a random variable L, returns a set S of size $|S| = O(\log(n)/\varepsilon^2)$ such that (L, S) is a $(\delta, 3)$-digest of $U := \bigcup_{X \in \mathcal{X}} X$ for $\delta := \frac{2}{n^3} + \mathbb{P}[L \notin \text{level}(U) \pm 1]$. The algorithm runs in time $O(m \log(n)/\varepsilon^2)$.

Algorithm 1: Digesting a decremental union

```
function erase-resample(𝒳, L)
    S := ∅
    foreach X ∈ 𝒳 do
        foreach x ∈ S do
            if X.contains(x) then
                remove x from S
        sample a random number N ~ Pois(X.size/2^L)
        if |S| + N > 160 log(n)/ε^2 then
            return ∅
        for k = 1, ..., N do
            x := X.sample()
            add x to S
    return S
```

Proof. The algorithm is described as Algorithm 1. The bound on the size $|S|$ is clear. The running time analysis is also easy: there are m iterations, each running in time $O(\log(n)/\varepsilon^2)$ due to the bound on $|S|$ (and on $|S| + N$). Next, we show the correctness. Assume first that the algorithm does not return from the if-clause. Let $X_1, \ldots, X_m$ be the objects in $\mathcal{X}$ in the order enumerated by the foreach-loop, and write $U := \bigcup_{i=1}^{m} X_i$. We define the random set $X_i^{(L)}$ as the set of points sampled from X_i in the for-k-loop; observe that $X_i^{(L)} \sim \text{Pois}(X_i, L)$ for every $i \in [n]$. Let S_i be the set S at the end of the iteration for $X = X_i$. Note that with $S_0 := \emptyset$ the algorithm computes $S_i = (S_{i-1} \setminus X_i) \cup X_i^{(L)}$ in each iteration. It follows that $S_i = \bigcup_{j=1}^{i} X_j^{(L)} \setminus (X_{j+1} \cup \cdots \cup X_i)$. Since the

random sets $X_j^{(L)} \setminus (X_{j+1} \cup \cdots \cup X_i) \sim \text{Pois}(X_j \setminus (X_{j+1} \cup \cdots \cup X_i), L)$ are L-samples with disjoint supports and the union of supports is $\bigcup_{j=1}^{i} X_j$, we have $S_i \sim \text{Pois}(\bigcup_{j=1}^{i} X_j, L)$. Now let us analyze the probability of the algorithm aborting. Observe that the algorithm aborts if and only if $|S_i| > 160 \log(n)/\varepsilon^2$ for some $i \in [n]$. By the monotonicity of the size of the support $\bigcup_{j=1}^{i} X_j$, we have $\mathbb{P}[|S_1| > 160 \log(n)/\varepsilon^2] \leq \cdots \leq \mathbb{P}[|S_n| > 160 \log(n)/\varepsilon^2]$. Fix $L \in \text{level}(U) \pm 1$. Since $(1+\varepsilon)\frac{\text{vol}(U)}{2^L} \leq (1+\varepsilon)\frac{\text{vol}(U)}{2^{\text{level}(U)-1}} \leq (1+\varepsilon)\frac{128 \log n}{\varepsilon^2} \leq \frac{160 \log n}{\varepsilon^2}$, we have

$$\mathbb{P}\left(|S_n| > \frac{160 \log n}{\varepsilon^2}\right) \leq \mathbb{P}\left(|S_n| > (1+\varepsilon)\frac{\text{vol}(U)}{2^L}\right) \leq \frac{2}{n^4},$$

where the second inequality follows from Lemma 5.3 by interpreting (L, S_n) as a size-1 ensemble of U and plugging in $V := U$. By a union bound over all events $|S_i| > 160 \log(n)/\varepsilon^2$, it follows that the algorithm aborts with probability at most $\frac{2}{n^3}$, if $L \in \text{level}(U) \pm 1$. If the algorithm does not abort, then we have shown above that it returns a set $S = S_n \sim \text{Pois}(\bigcup_{j=1}^{n} X_j, L) = \text{Pois}(U, L)$. It follows that we can compare the distribution of (L, S) with the size-3 ensemble $\mathcal{E} := \{(\ell, \text{Pois}(U, \ell)) \colon \ell \in \text{level}(U) \pm 1\}$. We can bound the error probability $\mathbb{P}[(L, S) \notin \mathcal{E}]$ by the probability that $L \notin \text{level}(U) \pm 1$ plus the probability that the algorithm aborts assuming $L \in \text{level}(U) \pm 1$, and we have bounded the latter probability by $\frac{2}{n^3}$. This proves the claim. □

5.2 Digesting decremental union

Next we study the decremental setting. That is, we receive a set of objects $\mathcal{X}$ and want to build a data structure that allows efficient deletions. At any time, it needs to quickly report a digest of the union of the surviving objects (i.e., objects that have not been deleted).

Lemma 5.6. Let $n \in \mathbb{N}$ and $\varepsilon \in (0, \frac{1}{16}]$ be parameters. Algorithm 2 supports initialize($\mathcal{X}$), which initializes with a multiset $\mathcal{X}$ of $m \leq n$ objects whose volumes are in $[(3n/\varepsilon)^2, (3n/\varepsilon)^4]$, and delete($X$), which removes object X from $\mathcal{X}$. The algorithm maintains a pair (L, S) which is a $(\frac{18}{n^2}, 6n)$-digest of $\bigcup_{X \in \mathcal{X}} X$ at any point in time, and $|S| = O(\log(n)/\varepsilon^2)$. In total, initialize and any sequence of delete operations run in expected time $O(m \log^2(n/\varepsilon)/\varepsilon^2)$. The

space complexity is $O(m + \log(n)/\varepsilon^2)$. Moreover, the set S only rarely gains new points: delete(X) returns true if the set S after the deletion is not a subset of the set S before the deletion (and false otherwise); and over any sequence of deletions only $O(\log(n/\varepsilon))$ deletions return true, with probability $1 - O(n^{-2})$.

Proof. Fix the initial objects and an arbitrary sequence of m deletions. After $t = 0, 1, \ldots, m$ deletions, let U_t be the union of the surviving objects, and let L_t, S_t be the variables L, S in the algorithm. We claim that (L_t, S_t) is a $(\frac{9(t+1)}{n^3}, 3(t+1))$-digest of U_t. Let us prove the claim by induction. First consider initialization ($t = 0$). By Theorem 5.1, with probability at least $1 - n^{-3}$ KLM computes $v \in (1 \pm 0.5)\,\mathrm{vol}(U_0)$, and in this case the algorithm computes $L \in \mathrm{level}(U_0) \pm 1$. Thus, erase-resample returns a $(\frac{3}{n^3}, 3)$-digest (L_0, S_0) of U_0 by Lemma 5.5. Now we proceed to $t \geq 1$. By the induction hypothesis, (L_{t-1}, S_{t-1}) is a $(\frac{9t}{n^3}, 3t)$-digest of U_{t-1}. Let $\mathcal{E}$ be a support ensemble of (L_{t-1}, S_{t-1}). Define

$$\mathcal{E}' := \{(\ell, R \cap U_t) : (\ell, R) \in \mathcal{E},\ \ell \leq \mathrm{level}(U_t) + 1\}.$$

Observe that $|\mathcal{E}'| \leq |\mathcal{E}| \leq 3t$ and $\mathcal{E}'$ is an ensemble of U_t since $\mathrm{level}(U_t) \leq \mathrm{level}(U_{t+1})$. Let E be the event that $(L_{t-1}, S_{t-1}) \in \mathcal{E}$. Let F be the event that $\left|2^{\ell}|R \cap U_t| - \mathrm{vol}(U_t)\right| \leq \varepsilon\,\mathrm{vol}(U_{t-1})$ for all $(\ell, R) \in \mathcal{E}$. Note that $\mathbb{P}(\overline{E}) \leq \frac{9t}{n^3}$ by the induction hypothesis, and $\mathbb{P}(\overline{F}) \leq \frac{2 \cdot 3t}{n^4} < \frac{6}{n^3}$ by Lemma 5.3. Assuming $E \cap F$, we have $L_{t-1} \in \mathrm{level}(U_{t-1}) \pm 1$ and $2^{L_{t-1}}|S_{t-1} \cap U_t| \in \mathrm{vol}(U_t) \pm \varepsilon\,\mathrm{vol}(U_{t-1})$. Let us distinguish three cases and derive further consequences:

(1) If $L_{t-1} \leq \mathrm{level}(U_t)$, then

$$|S_{t-1} \cap U_t| \geq \frac{\mathrm{vol}(U_t)}{2^{\mathrm{level}(U_t)}} - \varepsilon \frac{\mathrm{vol}(U_{t-1})}{2^{\mathrm{level}(U_{t-1})-1}} \geq \frac{32 \log n}{\varepsilon^2} - \varepsilon \frac{128 \log n}{\varepsilon^2} \geq \frac{24 \log n}{\varepsilon^2}.$$

So the algorithm does not refresh. As a result, $L_t = L_{t-1} \leq \mathrm{level}(U_t)$ and $S_t = S_{t-1} \cap U_t$, thus $(L_t, S_t) \in \mathcal{E}'$.

(2) If $L_{t-1} \geq \mathrm{level}(U_t) + 2$, then

$$|S_{t-1} \cap U_t| \leq \frac{\mathrm{vol}(U_t)}{2^{\mathrm{level}(U_t)+2}} + \varepsilon \frac{\mathrm{vol}(U_{t-1})}{2^{\mathrm{level}(U_{t-1})-1}} < \frac{16 \log n}{\varepsilon^2} + \varepsilon \frac{128 \log n}{\varepsilon^2} \leq \frac{24 \log n}{\varepsilon^2}.$$

So the algorithm refreshes. During refresh, the algorithm computes $L' \in \mathrm{level}(U_t) \pm 1$ with probability at least $1 - \frac{1}{n^3}$ by

Theorem 5.1, and then we have $L_t = \min\{L_{t-1} - 1, L'\} = L' \in \text{level}(U_t) \pm 1$. Hence, it returns a $(\frac{3}{n^3}, 3)$-digest (L_t, S_t) of U_t by Lemma 5.5.

(3) Otherwise $L_{t-1} = \text{level}(U_t) + 1$, and the algorithm may or may not refresh. If it does, then (L_t, S_t) is a $(\frac{3}{n^3}, 3)$-digest of U_t as in case (2). If it does not, then $L_t = L_{t-1} = \text{level}(U_t) + 1$ and $S_t = S_{t-1} \cap U_t$, thus $(L_t, S_t) \in \mathcal{E}'$.

Taking all cases into account, recalling $\mathbb{P}(\overline{E \cap F}) \leq \mathbb{P}(\overline{E}) + \mathbb{P}(\overline{F}) \leq \frac{9t+6}{n^3}$, and adding the error probability $\frac{3}{n^3}$ of the new digest after refreshing, we conclude that (L_t, S_t) is a $(\frac{9(t+1)}{n^3}, 3(t+1))$-digest of U_t. (In particular, it is a $(\frac{18}{n^2}, 6n)$-digest of the union of the remaining objects.) As a consequence, with probability at least $1 - O(n^{-2})$ we have $L_0 \in \text{level}(U_0) \pm 1$ and $L_{m-1} \in \text{level}(U_{m-1}) \pm 1$. On the other hand, $\text{level}(U_0) \leq O(\log(n/\varepsilon))$ and $\text{level}(U_{m-1}) \geq 0$ (because each object has volume in $[(3n/\varepsilon)^2, (3n/\varepsilon)^4]$), and every refresh causes L to decrease by at least one. So the number of calls to refresh is bounded by $O(\log(n/\varepsilon))$. Since we have $S_t \subseteq S_{t-1}$ except when we do a refresh, it follows that the set S can only gain points during $O(\log(n/\varepsilon))$ many deletions, with probability $1 - O(n^{-2})$. The size bound $|S| = O(\log(n)/\varepsilon^2)$ follows from Lemma 5.5. It remains to bound the running time and space usage. For initialize and refresh, the subroutine KLM runs in $O(m \log n)$ time by Theorem 5.1; the subroutine erase-resample runs in $O(m \log(n)/\varepsilon^2)$ time by Lemma 5.5; and the loop runs in $O(m \log(n)/\varepsilon^2)$ time since $|S| = O(\log(n)/\varepsilon^2)$ and each point in S is tested against at most m objects. For delete, we spend $O(\log(n)/\varepsilon^2)$ time in the loop. An extra $O(m \log(n)/\varepsilon^2)$ time may be incurred by a refresh. As we argued before, there are at most $O(\log(n/\varepsilon))$ refreshes throughout the deletion sequence with probability $1 - O(n^{-2})$. There can be at most m refreshes with the remaining probability $O(n^{-2})$. So the expected time for the whole deletion sequence is $O(m \log(n)/\varepsilon^2 \cdot (\log(n/\varepsilon) + m/n^2)) \leq O(m \log^2(n/\varepsilon)/\varepsilon^2)$. Finally, let us analyze the space complexity. Storing $\mathcal{X}$ in a binary search tree needs $O(m)$ space. Storing S and the counters m_x in a table needs $O(\log(n)/\varepsilon^2)$ space. Storing L needs constant space. So the algorithm uses $O(m + \log(n)/\varepsilon^2)$ space in total. □

5.3 Handling insertions

In this section, we apply the decremental data structure from the previous section to derive a dynamic algorithm for union volume estimation, using a novel variant of the logarithmic method [BS80]. Throughout the algorithm, the remaining objects are partitioned into bins $\mathcal{X}_0, \dots, \mathcal{X}_{\log n}$. Each bin $\mathcal{X}_j$ holds at most 2^j objects and is implemented by the decremental data structure. Every now and then, we merge some (small) bins into a larger bin by re-initializing the target bin with the objects in the source bins. Between any two merges that involve a bin $\mathcal{X}_j$, it undergoes deletions only. At any time, we can access a digest $(\mathcal{X}_j.L, \mathcal{X}_j.S)$ of the union $\bigcup_{X \in \mathcal{X}_j} X$. Additionally, for every point $x \in \mathcal{X}_j.S$, we keep track of the number $n_x := \#\{X \in \mathcal{X}_0 \cup \dots \cup \mathcal{X}_{j-1} : x \in X\}$.

Theorem 5.7. Let $n \in \mathbb{N}$ and $\hat{\varepsilon} \in (0, \frac{1}{16}]$ be parameters. Algorithm 3 maintains an initially empty multiset $\mathcal{X}$ of objects, where each object has volume in $[(3n/\hat{\varepsilon})^2, (3n/\hat{\varepsilon})^4]$, under updates insert($X$) and delete($X$) and query estimate(), which $(1 \pm \hat{\varepsilon})$-approximates the volume of the union of $\mathcal{X}$ and is correct with high probability. The amortized expected update time is $O(\log^5(n/\hat{\varepsilon})/\hat{\varepsilon}^2)$, the query time is $O(\log^4(n)/\hat{\varepsilon}^2)$, and the space complexity is $O(n + \log^4(n)/\hat{\varepsilon}^2)$.

Proof. Fix a sequence of at most n insertions and deletions. Then the partition of surviving objects into bins $\mathcal{X}_1, \dots, \mathcal{X}_{\log n}$ is determined; in particular $U_j := \bigcup_{X \in \mathcal{X}_j} X$ is determined, and we write $V_j := U_j \setminus (U_0 \cup \dots \cup U_{j-1})$. Note that the union of the surviving objects $U := \bigcup_{j=0}^{\log n} U_j = \bigcup_{j=0}^{\log n} V_j$ is a disjoint union of V_j's, thus $\mathrm{vol}(U) = \sum_{j=0}^{\log n} \mathrm{vol}(V_j)$. Now let us consider a call to estimate. By Lemma 5.6, each S_j is a $(\frac{18}{n^2}, 6n)$-digest of U_j. Note that the algorithm computes $R_j := V_j \cap S_j$, so by Corollary 5.4, we have $2^{L_j}|R_j| \in \mathrm{vol}(V_j) \pm \varepsilon\,\mathrm{vol}(U_j)$ with probability at least $1 - \frac{18}{n^2} - \frac{12n}{n^4} = 1 - O(n^{-2})$. By a union bound, this holds for all j simultaneously with probability $1 - O(n^{-1})$. Under this event,

$$\sum_{j=0}^{\log n} 2^{L_j}|R_j| \in \mathrm{vol}(U) \pm \varepsilon \sum_{j=0}^{\log n} \mathrm{vol}(U_j).$$

We can crudely bound the additive error by $\varepsilon(1 + \log n)\,\mathrm{vol}(U) = \hat{\varepsilon}\,\mathrm{vol}(U)$, just as desired. Next we analyze the running time. We

will charge the cost of delete to insertions. Note that in delete, it takes time $O(\log^2(n))$ to compute the bin j containing X, if for each j we store (pointers to) all objects in $\mathcal{X}_j$ in a binary search tree. The last for-loop takes time $O(\log^2(n)/\varepsilon^2)$. To bound the running time of the "if refreshed" block, we change our viewpoint and focus on the sequence of deletions occurring between two merges of a particular bin $\mathcal{X}_j$.

- The total cost of $\mathcal{X}_j$.delete is $O(2^j \log^2(n/\varepsilon)/\varepsilon^2)$ in expectation by Lemma 5.6.
- With probability $1 - O(n^{-2})$, there are $O(\log(n/\varepsilon))$ refreshes between two merges of bin $\mathcal{X}_j$ by Lemma 5.6, so the if-block is executed $O(\log(n/\varepsilon))$ times. Each time, the algorithm recomputes n_x for every $x \in S_j$ by testing against at most $\sum_{k=0}^{j-1} 2^k \le 2^j$ objects, which takes time $O(2^j \log(n)/\varepsilon^2)$. Therefore, the total contribution of the if-block is $O(2^j \log^2(n/\varepsilon)/\varepsilon^2)$ in expectation.

Therefore, over the sequence of deletions between two merges of $\mathcal{X}_j$, deletions from $\mathcal{X}_j$ take expected total time $O(2^j \log^2(n/\varepsilon)/\varepsilon^2)$. Note that at least 2^j insertions occur between two merges of bin $\mathcal{X}_j$. We can thus charge the cost of deletions to insertions. Each insertion is charged $O(\log n)$ times (once for every bin), so we charge each insertion with a cost of $O(\log^3(n/\varepsilon)/\varepsilon^2)$ in expectation. Now we analyze the running time of insert. It depends on the value h. By Lemma 5.6, the merge step costs $O\left(\sum_{j=0}^{h} 2^j \log^2(n/\varepsilon)/\varepsilon^2\right) = O\left(2^h \log^2(n/\varepsilon)/\varepsilon^2\right)$, which can be expensive in the worst case. However, bin $\mathcal{X}_h$ is merged at most once every 2^h updates, so it contributes amortized cost $O(\log^2(n/\varepsilon)/\varepsilon^2)$ to each insertion. Summing over all h, the amortized cost of merges is $O(\log^3(n/\varepsilon)/\varepsilon^2)$ per insertion. Next we consider the foreach-loops. Since each digest contains $O(\log(n)/\varepsilon^2)$ points by Lemma 5.6, and the algorithm spends constant time on each point, these loops take time $O(\log^2(n)/\varepsilon^2)$. To summarize, insert runs in amortized expected time $O(\log^3(n/\varepsilon)/\varepsilon^2) = O(\log^5(n/\varepsilon)/\hat{\varepsilon}^2)$ (and delete runs in amortized time 0). The running time of estimate is clearly $O(\log^2(n)/\varepsilon^2) = O(\log^4(n)/\hat{\varepsilon}^2)$. The space of bin $\mathcal{X}_j$ is $O(2^j + \log(n)/\varepsilon^2)$ by Lemma 5.6, and we can charge the space of counters n_x to it. In total the space complexity is

$O\left(\sum_{j=0}^{\log n}(2^j + \log(n)/\varepsilon^2)\right) = O(n + \log^4(n)/\hat{\varepsilon}^2)$. □

Theorem 2.1 follows by combining Theorem 5.7 with the "moreover" part in Lemma 4.1. Neither the space bound nor the time bound depends on the aspect ratio of the input.

6 Streaming Algorithm With Insertions and Suffix Queries

This section presents our streaming algorithm with insertions and suffix queries (Algorithm 4). In principle, we could use the language of digests (Definition 5.2) to analyze it. But here we choose to apply a simpler lemma that avoids the heavy machinery:

Lemma 6.1. Let U be a measurable set with $\text{level}(U) \in \{a, a+1, \dots, b\}$ and consider (possibly dependent) random sets $S^{(a)}, S^{(a+1)}, \dots, S^{(b)}$ where each $S^{(\ell)}$ is an ℓ-sample of U. Define a random variable $L := \min\left\{\ell \in \{a, \dots, b\} : |S^{(\ell)}| \le \frac{100 \log n}{\varepsilon^2}\right\}$. Then with probability $1 - O(n^{-1})$ we have $\text{level}(U) - 2 \le L \le \text{level}(U)$ and $2^L |S^{(L)}| \in (1 \pm \varepsilon)\,\text{vol}(U)$.

Proof. For each $a \le \ell \le \text{level}(U)$, define two events

$$E_\ell := \left\{2^\ell |S^{(\ell)}| \in (1 \pm \varepsilon)\,\text{vol}(U)\right\} \quad \text{and} \quad F_\ell := \left\{2^\ell |S^{(\ell)}| \in (1 \pm 0.5)\,\text{vol}(U)\right\}.$$

Then $\mathbb{P}(\overline{E_\ell}) \le O(n^{-8})$ by Lemma 4.4. Similarly as in Lemma 4.4 we can bound

$$\begin{aligned}\mathbb{P}(\overline{F_\ell}) &= \mathbb{P}\left(\left|2^\ell |S^{(\ell)}| - \text{vol}(U)\right| > \tfrac{1}{2}\,\text{vol}(U)\right) \\ &\le 2\exp\left(\frac{-(1/2)^2\,\text{vol}(U)}{4 \cdot 2^\ell}\right) \le 2\exp\left(\frac{-2\log(n) \cdot 2^{\text{level}(U)}}{2^\ell \cdot \varepsilon^2}\right) \le O(n^{-2^{\text{level}(U)-\ell}}),\end{aligned}$$

where we used the inequality $\frac{32 \log n}{\varepsilon^2} \le \frac{\text{vol}(U)}{2^{\text{level}(U)}}$ from the definition of $\text{level}(U)$, and $\varepsilon \le 1$. A union bound now shows that all events F_ℓ for $\ell = a, \dots, \text{level}(U)$ simultaneously hold with probability at least $1 - O(n^{-1})$. Hence, we have

$$\mathbb{P}\left(\bigcap_{\ell=\text{level}(U)-2}^{\text{level}(U)} E_\ell \cap \bigcap_{\ell=a}^{\text{level}(U)} F_\ell\right) \ge 1 - O(n^{-1}).$$

Conditioned on this event,

- For all $a \le \ell \le \text{level}(U) - 3$, $\left|S^{(\ell)}\right| \ge 0.5 \cdot \frac{\text{vol}(U)}{2^\ell} \ge 0.5 \cdot 8 \cdot \frac{32 \log n}{\varepsilon^2} = \frac{128 \log n}{\varepsilon^2}$. Hence $L \ge \text{level}(U) - 2$.
- $\left|S^{\text{level}(U)}\right| \le 1.5 \cdot \frac{\text{vol}(U)}{2^{\text{level}(U)}} \le 1.5 \cdot \frac{64 \log n}{\varepsilon^2} \le \frac{96 \log n}{\varepsilon^2}$. Hence $L \le \text{level}(U)$.
- $2^\ell |S^{(\ell)}|$ yields the desired approximation for each $\text{level}(U) - 2 \le \ell \le \text{level}(U)$.

These together prove the claim. □

Theorem 6.2. Algorithm 4 implicitly maintains a sequence of objects $X_1, \ldots, X_t$ where each object has volume in $[(3n/\varepsilon)^2, (3n/\varepsilon)^4]$. The algorithm supports update insert(X), which is given oracle access to the object X and appends X (as X_{t+1}) to the sequence, and query estimate(s), which outputs a $(1 \pm \varepsilon)$-approximation to $\text{vol}(X_s \cup \cdots \cup X_t)$ with high probability. The update time and space complexity are $O(\log(n/\varepsilon) \log(n)/\varepsilon^2)$, and the query time is $O(\log(n)/\varepsilon^2)$.

Proof. Since each object has volume in $[(3n/\varepsilon)^2, (3n/\varepsilon)^4]$, their union has volume in $[(3n/\varepsilon)^2, (3n/\varepsilon)^5]$, so the level of the union (as in Definition 4.3) is in $[0, 5 \log(3n/\varepsilon)]$. In particular, the correct level is considered by some choice of ℓ in the algorithm. Let us denote $U[a, b] := \bigcup_{i=a}^{b} X_i$, with the understanding that $U[a, b] = \emptyset$ for $a > b$. For the sake of analysis, we define the following random sets for $\ell = 0, \ldots, 5 \log(3n/\varepsilon)$:

- $X_i^{(\ell)} \sim \text{Pois}(X_i, \ell)$ for every $i \in [n]$.
- $U^{(\ell)}[a, b] := \bigcup_{i=a}^{b} (X_i^{(\ell)} \setminus U[i+1, b])$ for every $a \le b$.

Observe that $U^{(\ell)}[a, b]$ is a union of independent ℓ-samples $X_i^{(\ell)} \setminus U[i+1, b] \sim \text{Pois}(X_i \setminus U[i+1, b], \ell)$, whose supports are mutually disjoint, and the union of supports is $U[a, b]$. Therefore $U^{(\ell)}[a, b] \sim \text{Pois}(U[a, b], \ell)$. Also note that $U^{(\ell)}[1, t] \supseteq \cdots \supseteq U^{(\ell)}[t, t]$. Pretending that the for-i-loop of insert is always executed, we may interpret $X_i^{(\ell)}$ as the set of points sampled at iteration ℓ of insert(X_i). We claim two invariants over time:

(i) $s^{(\ell)} = \min\{a : |U^{(\ell)}[a,t]| \le C\}$;

(ii) $S^{(\ell)} = U^{(\ell)}[s^{(\ell)}, t]$.

The claims apparently hold after initialization ($t = 0$), so we focus on the call insert(X_i) at time $t \ge 1$. Consider an arbitrary iteration ℓ of the outer loop. By the invariant, at the beginning of iteration ℓ we have $S^{(\ell)} = U^{(\ell)}[s^{(\ell)}, t-1]$. The foreach-loop removes all points in X_t, so at the end of the foreach-loop we have $S^{(\ell)} = U^{(\ell)}[s^{(\ell)}, t-1] \setminus X_t$. Next $N \sim \text{Pois}(X_t.\text{size}/2^\ell)$ is sampled, and we interpret it as determining $|X_t^{(\ell)}|$ beforehand—the actual $X_t^{(\ell)}$ is realized later in the for-i-loop. Observe that for any $s^{(\ell)} \le a \le t$ we have

$$\begin{aligned}
U^{(\ell)}[a,t] &= X_t^{(\ell)} \cup (U^{(\ell)}[a,t-1] \setminus X_t) \\
&= X_t^{(\ell)} \cup ((U^{(\ell)}[s^{(\ell)},t-1] \cap U[a,t-1]) \setminus X_t) \\
&= X_t^{(\ell)} \cup ((U^{(\ell)}[s^{(\ell)},t-1] \setminus X_t) \cap U[a,t-1]) \\
&= X_t^{(\ell)} \cup (S^{(\ell)} \cap U[a,t-1]) \\
&= X_t^{(\ell)} \cup \{x \in S^{(\ell)} : \text{time}(X) \ge a\},
\end{aligned}$$

and thus $|U^{(\ell)}[a,t]| = N + |\{x \in S^{(\ell)} : \text{time}(X) \ge a\}|$. It follows that the binary search in the if-block correctly updates $s^{(\ell)}$ according to invariant (i). It also updates $S^{(\ell)}$ to still satisfy $S^{(\ell)} = U^{(\ell)}[s^{(\ell)}, t-1] \setminus X_t$ (for the new value of $s^{(\ell)}$). The for-i-loop now samples $X_t^{(\ell)}$ of size $N = |X_t^{(\ell)}|$ and adds it to $S^{(\ell)}$, so we obtain $S^{(\ell)} = U^{(\ell)}[s^{(\ell)}, t]$, so invariant (ii) holds. Correctness in the else-case also clearly holds. The running time of insert is easy to bound. Consider a fixed iteration ℓ of some insertion. The foreach-loop takes time $O(|S^{(\ell)}|) = O(C)$, due to invariants (i) and (ii). If we store the set $S^{(\ell)}$ in a binary search tree ordered by time(x), then the binary search takes time $O(\log n)$, and in the same time bound we can remove all items with time(x) $< a$. The for-i-loop takes time $O(N)$ and is only executed if $N \le C$, so it runs in time $O(C)$. As there are $O(\log(n/\varepsilon))$ iterations over ℓ, insert runs in time $O(\log(n/\varepsilon) \cdot (C + \log n)) = O(\log(n/\varepsilon)\log(n)/\varepsilon^2)$. To show the correctness of estimate(s) at time t we make two more claims:

(iii) $L = \min\{\ell : |U^{(\ell)}[s,t]| \le C\}$;

(iv) $R = U^{(L)}[s,t]$.

Inspecting estimate(s) we have $L = \min\{\ell : s^{(\ell)} \le s\}$. We have $s^{(\ell)} \le s$ if and only if $\left|U^{(\ell)}[s,t]\right| \le C$, due to (i) and the monotonicity $\left|U^{(\ell)}[1,t]\right| \ge \cdots \ge \left|U^{(\ell)}[s,t]\right|$; this proves (iii). Inspecting estimate(s) we have $R = \{x \in S^{(L)} : \text{time}(x) \ge s\} = S^{(L)} \cap U[s,t]$. Since $S^{(L)} = U^{(L)}[s^{(L)},t]$ by (ii), and $s^{(L)} \le s$, we obtain (iv). With the claims established, we apply Lemma 6.1 on the (dependent) random sets $U^{(\ell)}[s,t]$ for $0 \le \ell \le 5\log(3n/\varepsilon)$. Due to (iii), our L is defined exactly as in the lemma. The lemma implies that $2^L|U^{(L)}[s,t]|$ is a $(1 \pm \varepsilon)$-approximation to $|U[s,t]|$ with high probability. Since $|U^{(L)}[s,t]| = |R|$ by (iv), we return a correct estimate with high probability. Clearly, the running time of estimate is proportional to $O(\log(n/\varepsilon) + C) \le O(\log(n)/\varepsilon^2)$. Finally, the space complexity is $O(\log(n/\varepsilon) \cdot C) = O(\log(n/\varepsilon)\log(n)/\varepsilon^2)$. □

Theorem 2.2 now follows by combining Theorem 6.2 with Lemma 4.1. We point out that Algorithm 4 specializes to an insertion-only streaming algorithm if we hardwire $s = 1$ in the queries. In fact, the streaming algorithm by Meel, Vinodchandran, and Chakraborty [MVC21] can be viewed as a space-optimized version of Algorithm 4. Some subtle inaccuracies of their proof were addressed in our proof above.

7 Streaming Algorithm for Convex Bodies With Insertions and Deletions

In this section we present a streaming algorithm for convex bodies in $\mathbb{R}^d$ under insertions and deletions. Throughout this section we assume that d is constant. We also assume the following input condition in line with the literature on volume estimation. Let $R \ge r > 0$ be given. We assume that every object is convex, is contained in $[0,R]^d$, and contains a ball of radius r. The balls for different objects may differ, and we do not know them a priori. The objects are accessed via oracles as before. For $\alpha > 0$ and a convex body X with center o, we define the α-dilation of X by $\alpha X := \{o + \alpha(x - o) : x \in X\}$. Unless otherwise stated, we take o as the center of gravity.

7.1 Reduction to point counting

Let us introduce a related problem called union point counting: Given a parameter $\Delta \in \mathbb{N}$ and a multiset of objects $\mathcal{X}$ in $\mathbb{R}^d$ in the oracle model, compute a $(1 \pm \varepsilon)$-approximation to the cardinality $\left|[\Delta]^d \cap \bigcup_{X \in \mathcal{X}} X\right|$. Just like union volume estimation, we study this problem in the dynamic setting where $\mathcal{X}$ undergoes insertions and deletions. As the parameter Δ is usually implied from the context, we denote $U^\diamond := [\Delta]^d \cap U$. To relate union point counting and union volume estimation, we need a lemma about convexity.

Lemma 7.1. Let $X \subset \mathbb{R}^d$ be a convex body that contains a ball of radius $r > 0$. Assume $0 < \delta \leq r/d$ and let $\partial_\delta X := \left\{x \in \mathbb{R}^d : \mathrm{dist}(x, \partial X) \leq \delta\right\}$ be the δ-neighborhood of the boundary ∂X. Then we have $\mathrm{vol}(\partial_\delta X) \leq (2\mathrm{e}d\delta/r) \cdot \mathrm{vol}(X)$.

Proof. We take the center of the r-ball in X as the center of X. Consider the sets $S_{\text{in}} := \partial_\delta X \cap X$ and $S_{\text{out}} := \partial_\delta X \setminus X$. It is obvious that $\mathrm{vol}(S_{\text{in}}) \leq \mathrm{vol}(S_{\text{out}})$ because X is convex. (For a proof, it suffices to consider convex polytopes as they can approximate convex bodies within arbitrary precision. For each cone formed by the center and a facet of the polytope, the outer volume is larger than the inner volume.) So it remains to bound $\mathrm{vol}(S_{\text{out}})$. To this end, observe that $S_{\text{out}} \subseteq (X + (\delta/r)X) \setminus X$ where the addition is the Minkowski sum. Indeed, $(\delta/r)X$ contains a δ-ball by definition, so the Minkowski sum covers the neighborhood $\partial_\delta X$. Since X is convex, we have $X + (\delta/r)X = (1 + \delta/r)X$. Hence, we can bound

$$\begin{aligned}
\mathrm{vol}(S_{\text{out}}) &\leq \mathrm{vol}((1+\delta/r)X) - \mathrm{vol}(X) \\
&= ((1+\delta/r)^d - 1)\,\mathrm{vol}(X) \\
&\leq (1+\delta/r)^d \left(1 - (1-\delta/r)^d\right) \mathrm{vol}(X) \\
&\leq (1+\delta/r)^d \cdot (d\delta/r) \cdot \mathrm{vol}(X) \\
&\leq (\mathrm{e}d\delta/r) \cdot \mathrm{vol}(X),
\end{aligned}$$

where the second last step applied a union bound, and the last step used $\delta \leq r/d$ and the inequality $1 + x \leq e^x$. The lemma follows after taking S_{in} into account. □

Lemma 7.2. Assume that each input object is a convex body in $[0, R]^d$ and contains a ball of radius $r > 0$. If there is a dynamic

algorithm for union point counting with update and query time $T(n, \Delta, \varepsilon)$ and space $S(n, \Delta, \varepsilon)$, then there is a dynamic algorithm for union volume estimation with update and query time $O(T(n, \Delta', \frac{\varepsilon}{3}))$ and space $O(S(n, \Delta', \frac{\varepsilon}{3}))$ where $\Delta' = O(\frac{nR}{\varepsilon r})$.

Proof. We scale up the space (and implicitly the objects) by a factor of $\lambda := \frac{18d^{3/2}n}{\varepsilon r}$ in each dimension. So now each object is a convex body in $[0, \lambda R]^d$ and contains a ball of radius $\lambda r = \frac{18d^{3/2}n}{\varepsilon}$. We subdivide the space into unit cubes such that the centers of the cubes lie on the integer grid $[\Delta]^d$, where $\Delta := \lambda R + 1$. We build a dynamic data structure for union point counting on $[\Delta]^d$ with precision parameter $\varepsilon/3$. The insertions and deletions of objects are directly passed to the data structure. To estimate the union volume of the surviving objects, we query the data structure and obtain a count Z. We output Z/λ^d. The time and space complexity are clearly as stated. To show correctness, let $X_1, \ldots, X_n$ be the scaled objects currently in $\mathcal{X}$. For each $i \in [n]$, let X_i' be the union of cubes whose centers are in X_i. Denote $U := \bigcup_{i=1}^n X_i$ and $U' := \bigcup_{i=1}^n X_i'$. Since the counting data structure is correct, $Z \in (1 \pm \frac{\varepsilon}{3})|U^\diamond|$. On the other hand, $|U^\diamond| = \mathrm{vol}(U')$. It remains to relate $\mathrm{vol}(U)$ and $\mathrm{vol}(U')$. To this end, we write $\oplus$ for the symmetric difference and crudely bound

$$\mathrm{vol}(U \oplus U') = \mathrm{vol}\left(\left(\bigcup_{i=1}^n X_i\right) \oplus \left(\bigcup_{i=1}^n X_i'\right)\right) \le \sum_{i=1}^n \mathrm{vol}(X_i \oplus X_i'). \qquad (\star)$$

We claim that $\mathrm{vol}(X_i \oplus X_i') \le \frac{\varepsilon}{3n}\mathrm{vol}(X_i)$ for all $i \in [n]$. Indeed, the symmetric difference $X_i \oplus X_i'$ is contained in the neighborhood $\partial_\delta X_i$ for $\delta := \sqrt{d}$. Since $\delta \le (\lambda r)/d$, we can apply Lemma 7.1 to bound $\mathrm{vol}(\partial_\delta X_i) \le \frac{2\mathrm{e}d\delta}{\lambda r}\mathrm{vol}(X_i) < \frac{\varepsilon}{3n}\mathrm{vol}(X_i)$. Now we can continue to bound

$$(\star) \le \frac{\varepsilon}{3n}\sum_{i=1}^n \mathrm{vol}(X_i) \le \frac{\varepsilon}{3n}\sum_{i=1}^n \mathrm{vol}(U) = \frac{\varepsilon}{3}\mathrm{vol}(U).$$

Therefore, $\mathrm{vol}(U') \in (1 \pm \frac{\varepsilon}{3})\mathrm{vol}(U)$, and consequently $Z \in (1 \pm \frac{\varepsilon}{3})^2\mathrm{vol}(U) \subset (1 \pm \varepsilon)\mathrm{vol}(U)$. □

Due to Lemma 7.2, we focus on solving union point counting in what follows.

7.2 Preparations

Theorem 7.3 (John ellipsoid [Joh48])**.** Let $X \subset \mathbb{R}^d$ be a convex body. There exists a unique ellipsoid $E \subseteq X$ of maximum volume, and it satisfies $X \subseteq 2d^{3/2}E$. Similarly, there exists a unique ellipsoid $E' \supseteq X$ of minimum volume, and it satisfies $X \supseteq \frac{1}{2d^{3/2}}E'$. The sets E and E' are called the inner and outer John ellipsoids of X, respectively.

Theorem 7.4 (Welzl [Wel05])**.** There is an algorithm that computes the minimum enclosing ellipsoid of k points in $\mathbb{R}^d$ in $O(k)$ expected time for constant d.

Corollary 7.5. Given a convex body $X \subset \mathbb{R}^d$, we can compute in expected time $O(\log n)$ a rotated box $B \supseteq X$ such that $\text{vol}(B) \leq (12d^4)^d \,\text{vol}(X)$.

Proof. The algorithm is simple: Sample $k := (6d^2)^d(d + 2\log n)$ random points $x_1, \ldots, x_k \in X$, compute the minimum enclosing ellipsoid F using Theorem 7.4,[4] then output the minimum bounding box B of the dilated ellipsoid $6d^2F$. The running time is clearly $O(k) = O(\log n)$ since d is a constant. To show correctness, let E be the inner John ellipsoid of X, and let G be a box of maximum volume contained in E. (If E is a ball, then G is a cube. In general G is a cube stretched along the axes of E.) We divide each side of G evenly into three segments. This induces 3^d grid cells in G, of which 2^d cells are at the corners. We denote the cell at the center by G_0. See Figure 1. Note that $G = 3G_0$ and $E \subseteq \sqrt{d}G$ by definition. Additionally, $X \subseteq 2d^{3/2}E$ by Theorem 7.3. Chaining these together, we obtain $X \subseteq 6d^2G_0$. In particular, the volume of each cell is at least $\rho := \frac{1}{(6d^2)^d}$ times the volume of X. Now consider a specific corner cell. For each $i \in [k]$, the sample point x_i hits the cell with probability at least ρ. So the probability that no sample point hits the cell is at most

$$(1-\rho)^k \leq \exp(-\rho k) = \mathrm{e}^{-(d+2\log n)} = \mathrm{e}^{-d} \cdot n^{-2}.$$

Applying a union bound over all corner cells, we have with probability at least $1 - n^{-2}$ that all corner cells are hit. Conditioned

[4] As an alternative to Welzl's algorithm, we can apply Barequet and Har-Peled's approximate bounding box algorithm [BH01] to these k random points.

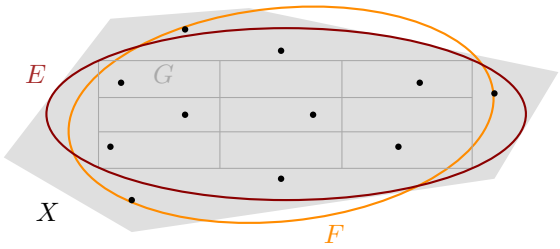


Figure 1: The convex body X, the inner John ellipsoid E, its inscribing box G, the subdivision into cells, and the minimum ellipsoid F of the sample points.

on this event, $G_0 \subseteq F$ and thus $X \subseteq 6d^2F \subseteq B$. It remains to relate $\mathrm{vol}(B)$ to $\mathrm{vol}(X)$. To this end, we consider the outer John ellipsoid E' of X. We have $\mathrm{vol}(E') \leq (2d^{3/2})^d \mathrm{vol}(X)$ by Theorem 7.3. Next note that $\mathrm{vol}(F) \leq \mathrm{vol}(E')$, for otherwise E' would be a smaller ellipsoid containing all sample points, contradicting the definition of F. Finally, $B \subseteq \sqrt{d} \cdot (6d^2F)$ by definition, thus $\mathrm{vol}(B) \leq (6d^{5/2})^d \mathrm{vol}(F)$. Chaining these together, we conclude that $\mathrm{vol}(B) \leq (12d^4)^d \mathrm{vol}(X)$. □

Theorem 7.6 (Lenstra [Len83])**.**[5] Let $k \in \mathbb{N}$ be a constant. There is an algorithm that solves any given integer linear program with k variables and description size m in time $O(m^{c_k})$, where c_k is a constant depending only on k.

Corollary 7.7. Let $k \in \mathbb{N}$ be a constant. There is an algorithm that lists all feasible solutions of any given linear program with k variables and description size m in time $O(z \cdot m^{c'_k})$, where z is the output size and c'_k is a constant depending only on k.

Proof. We describe a recursive algorithm list(P, k) that lists all integral points inside a k-dimensional polytope P. Denoting by $P[x_i \mapsto a]$ the polytope P after fixing the i-th coordinate to a constant a, the algorithm is given as Algorithm 5. We show by induction on k that list(P, k) is correct and runs in time $O_k(z \cdot m^{c'_k})$, where $O_k(\cdot)$ hides constant factors depending on k. In the base case $k = 0$, the polytope P is a singleton and thus $z = 1$. The algorithm correctly outputs the unique point in P in time $O(m) = O(z \cdot m)$. Now

[5] There were subsequent improved algorithms, see e.g., [RR23], but we don't need them here.

consider $k \geq 1$. Denote by $a_1 < \cdots < a_r < \infty$ the evolution of a during the while-loop. Observe that $a_1, \ldots, a_r$ enumerates all feasible integer assignments of the last coordinate x_k. For each $t \in [r]$, the algorithm recurses on the non-empty subpolytope $P_t := P[x_k \mapsto a_t]$. By induction, this lists all (say $z_t \geq 1$ many) integral points in P_t. Correctness follows since the polytopes $P_1, \ldots, P_r$ are disjoint and cover all integral points of P. Moreover, we have $z = \sum_{t=1}^{r} z_t$. To bound the running time, for each $t \in [r]$ the recursive call costs time $O_k(z_t \cdot (m+1)^{c'_{k-1}}) = O_k(z_t \cdot m^{c'_{k-1}})$ by induction, and updating a costs time $O(m^{c_k})$ by Theorem 7.6. So the total time is

$$\sum_{t=1}^{r} \left(O(m^{c_k}) + O_k(z_t \cdot m^{c'_{k-1}}) \right) = O_k(r \cdot m^{c_k} + z \cdot m^{c'_{k-1}}).$$

Note that $r \leq z$ since $z_1, \ldots, z_r \geq 1$. So the time is $O_k(z \cdot m^{c'_k})$ where $c'_k := \max(c_k, c'_{k-1})$. □

Theorem 7.8 (Sparse recovery, e.g., [FIS05, JWZ22])**.** Let $\delta \in (0,1)$ and $k \in \mathbb{N}$ be parameters, and let Ω be a finite universe. There is a data structure that implicitly maintains a vector $v = (v_x)_{x \in \Omega}$ using $O(k \operatorname{polylog}(|\Omega|/\delta))$ words of space. The vector is initially zero and the data structure supports two operations in time $O(\operatorname{polylog}(|\Omega|/\delta))$:

- update(x, b): given $x \in \Omega$ and $b \in \{-1, 1\}$, updates $v_x := v_x + b$.
- support(): returns the support size $s := \#\{x \in \Omega : v_x \neq 0\}$ if $s \leq k$, and ∞ if $s > k$. The answer is correct with probability at least $1 - \delta$.

7.3 Counting via weak sampling

We need a variant of ℓ-samples in the discrete setting:

Definition 7.9 (weak q-sample)**.** Let $U^\diamond$ be a finite set and $q \in [0,1]$. A random subset $S \subseteq U^\diamond$ is a weak q-sample of $U^\diamond$ if $\mathbb{P}(x \in S) = q$ for all $x \in U^\diamond$, and the events $\{x \in S\}, \{y \in S\}$ are independent for all pairs of distinct $x, y \in U^\diamond$.

Lemma 7.10. If S is a weak q-sample of $U^\diamond$ with $q \geq \frac{20}{\varepsilon^2 |U^\diamond|}$, then $|S|/q \in (1 \pm \varepsilon)|U^\diamond|$ with probability at least $19/20$.

Proof. We bound the expectation and variance of $|S|$. For each $x \in U^\diamond$ we define a random variable I_x indicating the event $\{x \in S\}$. Clearly $|S| = \sum_{x \in U^\diamond} I_x$, so $\mathbb{E}\,|S| = q|U^\diamond|$. Since the indicator variables are pairwise independent, $\operatorname{Var}|S| = \sum_{x \in U^\diamond} \operatorname{Var} I_x = q(1-q)|U^\diamond| \le \mathbb{E}\,|S|$. Applying Chebyshev's inequality,

$$\mathbb{P}\left(\left|\frac{|S|}{q} - |U^\diamond|\right| \ge \varepsilon |U^\diamond|\right) \le \frac{\operatorname{Var}|S|}{(\varepsilon\,\mathbb{E}\,|S|)^2} \le \frac{1}{\varepsilon^2\,\mathbb{E}\,|S|} = \frac{1}{\varepsilon^2 q|U^\diamond|} \le \frac{1}{20}. \qquad \square$$

Let us explain the idea of our data structure. Fix a prime $p \in [\Delta^d, 2\Delta^d]$ and sample $a, b \in \mathbb{F}_p$ uniformly at random. Define a hash function $h_{a,b} : [\Delta]^d \to \mathbb{F}_p$ by

$$h_{a,b}(x_1, \ldots, x_d) := \left(a + b \sum_{i=1}^{d} \Delta^{i-1} x_i\right) \bmod p.$$

The data structure morally maintains $S^{(\ell)} := \{x \in U^\diamond : h_{a,b}(x) \le \lceil p/2^\ell \rceil - 1\}$ for each $\ell \in \mathbb{N}$, where U is the union of the surviving objects. Note that $\mathbb{P}(x \in S^{(\ell)}) = \lceil p/2^\ell \rceil / p =: q_\ell$, and the events $\{x \in S^{(\ell)}\}, \{y \in S^{(\ell)}\}$ are pairwise independent for $x \ne y$. So $S^{(\ell)}$ is a weak q_ℓ-sample of $U^\diamond$. We then try to apply Lemma 7.10 to retrieve an estimate of $|U^\diamond|$.

7.4 Weak sampling from a single object

Lemma 7.11. There is an algorithm filter(X, a, b, ℓ) that, given a convex body X, random numbers $a, b \in \mathbb{F}_p$, and an integer ℓ, computes $X^\diamond \cap S^{(\ell)} = \{x \in X^\diamond : h_{a,b}(x) \le \lceil p/2^\ell \rceil - 1\}$. It runs in time $O(\text{polylog}(\Delta) \cdot |X^\diamond|/2^\ell)$ in expectation over the randomness of a, b.

Proof. We describe the algorithm filter(X, a, b, ℓ) as follows.

1. Compute a bounding box $B \supseteq X$ with $\text{vol}(B) \le (12d^4)^d \,\text{vol}(X)$ by Corollary 7.5. Let $H_1, \ldots, H_{2d}$ be the defining hyperplanes such that $B = \{x \in \mathbb{R}^d : H_i(x) \ge 0, \forall i \in [2d]\}$.

2. Compute the set of solutions $\mathcal{S}$ to the following integer linear

program by Corollary 7.7.

$$\begin{aligned} \forall i \in [2d], \quad & H_i(x) \geq 0 \\ a + b \sum_{i=1}^{d} \Delta^{i-1} x_i &= y + zp \\ x &\in [\Delta]^d \\ y &\in \{0, \ldots, \lceil p/2^\ell \rceil - 1\} \\ z &\in \{0, \ldots, d\Delta^d\} \end{aligned}$$

Here, additions and multiplications are over $\mathbb{Z}$ instead of $\mathbb{F}_p$.

3. Return $\{x : (x, y, z) \in \mathcal{S}$ and $X.\text{contains}(x)\}$.

The correctness proof is easy. For $x \in [\Delta]^d$, we write $\eta_{a,b}(x) := a + b\sum_{i=1}^{d} \Delta^{i-1} x_i$ and note that $0 \leq \eta_{a,b}(x) < p + pd\Delta^d$. Hence, $h_{a,b}(x) = y$ if and only if $\eta_{a,b}(x) = y + zp$ for some (unique) $z \in \{0, \ldots, d\Delta^d\}$. So there is a bijection between $\{x \in B^\diamond : h_{a,b}(x) \leq \lceil p/2^\ell \rceil - 1\}$ and $\mathcal{S}$. Since $B^\diamond \supseteq X^\diamond$, the algorithm returns $\{x \in X^\diamond : h_{a,b}(x) \leq \lceil p/2^\ell \rceil - 1\}$. We next analyze the running time. Step 1 runs in $O(\log n)$ time by Corollary 7.5. For step 2, the ILP contains $d + 2$ variables and $2d + 1$ linear inequalities. So for constant d it runs in time $(\log \Delta)^{c'_{d+2}} = \text{polylog}(\Delta)$ per solution by Corollary 7.7. Observe that $\mathbb{E}\,|\mathcal{S}| = q_\ell |B^\diamond| = O(|B^\diamond|/2^\ell)$ over the randomness of a, b. So the expected total time is $O(\text{polylog}(\Delta)|B^\diamond|/2^\ell)$. Recall that $\text{vol}(B) \leq O(\text{vol}(X))$, and that $\frac{|B^\diamond|}{\text{vol}(B)}, \frac{|X^\diamond|}{\text{vol}(X)} \in [\frac{1}{2}, 2]$ by Lemma 7.1, so we have $|B^\diamond| \leq O(|X^\diamond|)$. Therefore, the running time is $O(\text{polylog}(\Delta) \cdot |X^\diamond|/2^\ell)$. □

7.5 Weak sampling from the union

Theorem 7.12. Assume $\varepsilon \in (0, \frac{1}{4}]$. Algorithm 6 satisfies the following correctness guarantee. Consider an arbitrary sequence of insert and delete operations, and let $\mathcal{X}$ be the multiset of surviving objects. With probability at least 0.7, estimate() returns a $(1 \pm \varepsilon)$-approximation to $|U^\diamond| = |[\Delta]^d \cap U|$ where $U = \bigcup_{X \in \mathcal{X}} X$. Moreover, the algorithm uses $O(\text{polylog}(\Delta)/\varepsilon^2)$ space and expected time per operation.

Proof. Let us define a hierarchy of random sets $S^{(0)} \supseteq S^{(1)} \supseteq \cdots \supseteq S^{\lfloor \log p \rfloor}$, where

$$S^{(\ell)} := \left\{ x \in U^\diamond : h_{a,b}(x) \le \lceil p/2^\ell \rceil - 1 \right\}.$$

Note that $S^{(\ell)}$ is a weak q_ℓ-sample of $U^\diamond$, where $q_\ell := \lceil p/2^\ell \rceil / p \in [2^{-\ell}, 2^{-\ell+1}]$. Since $q_\ell / q_r \in [2^{r-\ell-1}, 2^{r-\ell+1}]$ for all ℓ, r, there exists an integer $\ell^* \in \mathbb{N}$ such that $20/\varepsilon^2 \le q_{\ell^*}|U^\diamond| < 80/\varepsilon^2$. For $\ell \ge \ell^* - 4$, we make some observations on the sparse vectors v^ℓ in the algorithm. Every surviving object $X \in \mathcal{X}$ satisfies $X.\text{size} \le \text{vol}(U) \le (1+\varepsilon)|U^\diamond| \le \frac{5}{4} \cdot 80/(\varepsilon^2 q_{\ell^*}) \le 1600/(\varepsilon^2 q_\ell)$, which satisfies the if-condition at ℓ when X was inserted. The if-block thus increments the vector v^ℓ on coordinates $X \cap S^{(\ell)}$. On the other hand, every non-surviving object is treated consistently during insertion and deletion, so it does not contribute to v^ℓ. Therefore, $\text{supp}(v^\ell) = \bigcup_{X \in \mathcal{X}} (X \cap S^{(\ell)}) = S^{(\ell)}$. In particular, $\text{supp}(v^{\ell^*-4}) \supseteq \cdots \supseteq \text{supp}(v^{\lfloor \log p \rfloor})$. Now consider a call to estimate(). Let F be the event that all sparse vectors behave correctly according to Theorem 7.8. By a union bound, $\mathbb{P}(F) \ge 19/20$. We also define an event $E_\ell := \left\{ |S^{(\ell)}|/q_\ell \in (1 \pm \varepsilon)|U^\diamond| \right\}$ for $\ell^* - 4 \le \ell \le \ell^*$. Then $\mathbb{P}(E_\ell) \ge 19/20$ by Lemma 7.10. Under the event $F \cap \bigcap_{\ell=\ell^*-4}^{\ell^*} E_\ell$, we have

- $\left|S^{\ell^*-4}\right| \ge (1-\varepsilon) \cdot q_{\ell^*-4}|U^\diamond| \ge \frac{3}{4} \cdot \frac{20}{\varepsilon^2} \frac{q_{\ell^*-4}}{q_{\ell^*}} \ge \frac{120}{\varepsilon^2} > k$. So $v^{\ell^*-4}.\text{support}() = \infty$.
- $\left|S^{\ell^*}\right| \le (1+\varepsilon) \cdot q_{\ell^*}|U^\diamond| \le \frac{5}{4} \cdot \frac{80}{\varepsilon^2} = \frac{100}{\varepsilon^2} = k$. So $v^\ell.\text{support}() < \infty$ for all $\ell \ge \ell^*$.

So the algorithm computes $\ell^* - 3 \le L \le \ell^*$ and outputs $v^L.\text{support}()/q_L = |S^{(L)}|/q_L$, which is a $(1 \pm \varepsilon)$-approximation to $|U^\diamond|$. The success probability is thus at least $\mathbb{P}(F \cap \bigcap_{\ell=\ell^*-4}^{\ell^*} E_\ell) \ge \frac{19}{20} - 5 \cdot \frac{1}{20} = 0.7$. Next we analyze the complexity. The space is dominated by $\log p = O(\log \Delta)$ sparse vectors, each using $O(k \,\text{polylog}(\Delta))$ words by Theorem 7.8. So the space complexity is $O(\text{polylog}(\Delta)/\varepsilon^2)$. Regarding time complexity, each update has $\log p = O(\log \Delta)$ iterations, and each iteration runs in expected time $O(\text{polylog}(\Delta) \cdot 2^{-\ell} \cdot 2^\ell/\varepsilon^2) = O(\text{polylog}(\Delta)/\varepsilon^2)$ by Lemma 7.11. So the update time is $O(\text{polylog}(\Delta)/\varepsilon^2)$ as claimed. □

Now Theorem 2.3 follows directly from Theorem 7.12 and Lemma 7.2. (To amplify the success probability, we run $O(\log n)$ independent copies and output the median.)

A Fat Objects Are Easy

We briefly mention that for fat objects in a constant dimension, there are simple ways to solve the dynamic union volume estimation. To this end, we can build a quadtree and replace each object X by a union $\widetilde{X}$ of $O(1/\varepsilon^d)$ quadtree cells of side length $O(\varepsilon \cdot \mathrm{diam}(X))$ where $\mathrm{diam}(X)$ is the diameter of X. It is straightforward to maintain the volume of the union of all these quadtree cells, using a tree structure augmented with intermediate volumes stored at each node. (We may assume that the aspect ratio is polynomial in n due to Lemma 4.1 and so the tree height is $O(\log n)$.) For fat objects, it is not difficult to show that the volume of $\bigcup_{X \in \mathcal{X}} \widetilde{X}$ (which is contained in a union of a $(1+O(\varepsilon))$-factor expansion of X) gives a good approximation to the volume of $\bigcup_{X \in \mathcal{X}} X$. It may even be possible to adapt this approach in streaming settings by incorporating known streaming techniques. The problem becomes more nontrivial and subtle when the objects may not be fat: First, we cannot afford to replace a non-fat object with quadtree cells. Second, and more curiously, for non-fat objects, if we take some $(1+\varepsilon)$-factor expansion $\widetilde{X}$ of each object X, it is not obvious how the volume of $\bigcup_{X \in \mathcal{X}} \widetilde{X}$ relates to the volume of $\bigcup_{X \in \mathcal{X}} X$. In fact, this type of question (even in the special case when the objects are narrow "rods") is related to the infamous Kakeya set conjecture, for which considerable recent effort has been spent on just the 3D case [WZ25] (and much less is known in 4D and beyond)!

References

[AAS18] Pankaj K. Agarwal, Lars Arge, and Frank Staals. Improved dynamic geodesic nearest neighbor searching in a simple polygon. In Proc. 34th International Symposium on Computational Geometry (SoCG), volume 99 of LIPIcs, pages 4:1–4:14, 2018. doi:10.4230/LIPICS.SOCG.2018.4.

[ACS+22] Pankaj K. Agarwal, Hsien-Chih Chang, Subhash Suri, Allen Xiao, and Jie Xue. Dynamic geometric set cover

and hitting set. ACM Transactions on Algorithms, 18(4):40:1–40:37, 2022. doi:10.1145/3551639.

[BBD+02] Brian Babcock, Shivnath Babu, Mayur Datar, Rajeev Motwani, and Jennifer Widom. Models and issues in data stream systems. In Proc. 21st ACM Symposium on Principles of Database Systems (PODS), pages 1–16, 2002. doi:10.1145/543613.543615.

[BC25] Sujoy Bhore and Timothy M. Chan. Fast static and dynamic approximation algorithms for geometric optimization problems: Piercing, independent set, vertex cover, and matching. In Proc. 2025 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA), pages 2357–2386, 2025. doi:10.1137/1.9781611978322.79.

[BF10] Karl Bringmann and Tobias Friedrich. Approximating the volume of unions and intersections of high-dimensional geometric objects. Computational Geometry, 43(6-7):601–610, 2010. doi:10.1016/J.COMGEO.2010.03.004.

[BH01] Gill Barequet and Sariel Har-Peled. Efficiently approximating the minimum-volume bounding box of a point set in three dimensions. Journal of Algorithms, 38(1):91–109, 2001. doi:10.1006/JAGM.2000.1127.

[BLN+25] Karl Bringmann, Kasper Green Larsen, André Nusser, Eva Rotenberg, and Yanheng Wang. Approximating Klee's measure problem and a lower bound for union volume estimation. In Proc. 41st International Symposium on Computational Geometry (SoCG), pages 25:1–25:16, 2025. doi:10.4230/LIPICS.SOCG.2025.25.

[BNTW24] Sujoy Bhore, Martin Nöllenburg, Csaba D. Tóth, and Jules Wulms. Fully dynamic maximum independent sets of disks in polylogarithmic update time. In 40th International Symposium on Computational Geometry (SoCG), volume 293 of LIPIcs, pages 19:1–19:16, 2024. doi:10.4230/LIPICS.SOCG.2024.19.

[Bra16] Vladimir Braverman. Sliding window algorithms. In Ming-Yang Kao, editor, Encyclopedia of Algorithms, pages 2006–2011. Springer New York, New York, NY, 2016. doi:10.1007/978-1-4939-2864-4_797.

[BS80] Jon Louis Bentley and James B Saxe. Decomposable searching problems I. Static-to-dynamic transformation. Journal of Algorithms, 1(4):301–358, 1980. doi:10.1016/0196-6774(80)90015-2.

[Can19] Clément Canonne. A short note on Poisson tail bounds, 2019. URL: https://github.com/ccanonne/probabilitydistributiontoolbox/blob/master/poissonconcentration.pdf.

[CH24] Timothy M. Chan and Zhengcheng Huang. Dynamic geometric connectivity in the plane with constant query time. In 40th International Symposium on Computational Geometry (SoCG), volume 293 of LIPIcs, pages 36:1–36:13, 2024.

[Cha10] Timothy M. Chan. A (slightly) faster algorithm for Klee's measure problem. Computational Geometry, 43(3):243–250, 2010. doi:10.1016/J.COMGEO.2009.01.007.

[Cha13] Timothy M. Chan. Klee's measure problem made easy. In Proc. 54th Annual IEEE Symposium on Foundations of Computer Science (FOCS), pages 410–419, 2013. doi:10.1109/FOCS.2013.51.

[CHSX25] Timothy M. Chan, Qizheng He, Subhash Suri, and Jie Xue. Dynamic geometric set cover, revisited. SIAM Journal on Computing, 54(3):664–701, 2025. URL: https://doi.org/10.1137/23m1596582, doi:10.1137/23M1596582.

[CMV13] Supratik Chakraborty, Kuldeep S. Meel, and Moshe Y. Vardi. A scalable approximate model counter. In Proc. 19th International Conference on Principles and Practice of Constraint Programming (CP), pages 200–216, 2013. doi:10.1007/978-3-642-40627-0_18.

[CPR11] Timothy M. Chan, Mihai Pătraşcu, and Liam Roditty. Dynamic connectivity: Connecting to networks and geometry. SIAM Journal on Computing, 40(2):333–349, 2011. doi:10.1137/090751670.

[CS06] Timothy M. Chan and Bashir S. Sadjad. Geometric optimization problems over sliding windows. Int. J. Comput. Geom. Appl., 16(2-3):145–158, 2006. doi:10.1142/S0218195906001975.

[CTG+23] Nofar Carmeli, Nikolaos Tziavelis, Wolfgang Gatterbauer, Benny Kimelfeld, and Mirek Riedewald. Tractable orders for direct access to ranked answers of conjunctive queries. ACM Transactions on Database Systems, 48(1):1–45, 2023. doi:10.1145/3578517.

[CZB+20] Nofar Carmeli, Shai Zeevi, Christoph Berkholz, Benny Kimelfeld, and Nicole Schweikardt. Answering (unions of) conjunctive queries using random access and random-order enumeration. In Proc. 39th ACM Symposium on Principles of Database Systems (PODS), pages 393–409, 2020. doi:10.1145/3375395.3387662.

[DFK91] Martin E. Dyer, Alan M. Frieze, and Ravi Kannan. A random polynomial time algorithm for approximating the volume of convex bodies. Journal of the ACM, 38(1):1–17, 1991.

[DGIM02] Mayur Datar, Aristides Gionis, Piotr Indyk, and Rajeev Motwani. Maintaining stream statistics over sliding windows. SIAM Journal on Computing, 31(6):1794–1813, 2002. doi:10.1137/S0097539701398363.

[DS07] Nilesh N. Dalvi and Dan Suciu. Efficient query evaluation on probabilistic databases. The VLDB Journal, 16(4):523–544, 2007. doi:10.1007/S00778-006-0004-3.

[FIS05] Gereon Frahling, Piotr Indyk, and Christian Sohler. Sampling in dynamic data streams and applications.

In Proc. 21st ACM Symposium on Computational Geometry (SoCG), pages 142–149, 2005. doi:10.1145/1064092.1064116.

[FKZ05] Joan Feigenbaum, Sampath Kannan, and Jian Zhang. Computing diameter in the streaming and sliding-window models. Algorithmica, 41(1):25–41, 2005. doi:10.1007/S00453-004-1105-2.

[FSW25] Shiyuan Feng, William Swartworth, and David P. Woodruff. Tight bounds for heavy-hitters and moment estimation in the sliding window model. In Proc. 52nd International Colloquium on Automata, Languages, and Programming (ICALP), pages 75:1–75:19, 2025. doi:10.4230/LIPIcs.ICALP.2025.75.

[GK23] Egor Gorbachev and Marvin Künnemann. Combinatorial designs meet hypercliques: Higher lower bounds for Klee's measure problem and related problems in dimensions $d \geq 4$. In Proc. 39th International Symposium on Computational Geometry (SoCG), pages 36:1–36:14, 2023. doi:10.4230/LIPICS.SOCG.2023.36.

[HNW20] Monika Henzinger, Stefan Neumann, and Andreas Wiese. Dynamic approximate maximum independent set of intervals, hypercubes and hyperrectangles. In 36th International Symposium on Computational Geometry (SoCG), volume 164 of LIPIcs, pages 51:1–51:14, 2020. doi:10.4230/LIPICS.SOCG.2020.51.

[Ind04] Piotr Indyk. Algorithms for dynamic geometric problems over data streams. In Proc. 36th annual ACM Symposium on Theory of Computing, pages 373–380, 2004.

[Joh48] Fritz John. Extremum problems with inequalities as subsidiary conditions. In Studies and Essays Presented to R. Courant on his 60th Birthday. Interscience Publishers, Inc., New York, NY, 1948.

[JST11] Hossein Jowhari, Mert Saglam, and Gábor Tardos. Tight bounds for L_p samplers, finding duplicates

in streams, and related problems. In Proc. 30th ACM Symposium on Principles of Database Systems (PODS), pages 49–58, 2011. doi:10.1145/1989284.1989289.

[JWZ22] Rajesh Jayaram, David P. Woodruff, and Samson Zhou. Truly perfect samplers for data streams and sliding windows. In Proc. 41st ACM Symposium on Principles of Database Systems (PODS), pages 29–40, 2022. doi:10.1145/3517804.3524139.

[Kar01] David R. Karger. A randomized fully polynomial time approximation scheme for the all-terminal network reliability problem. SIAM Review, 43(3):499–522, 2001. doi:10.1137/S0036144501387141.

[KKS08] Benny Kimelfeld, Yuri Kosharovsky, and Yehoshua Sagiv. Query efficiency in probabilistic XML models. In Proc. ACM SIGMOD International Conference on Management of Data, pages 701–714, 2008. doi:10.1145/1376616.1376687.

[KL85] Richard M. Karp and Michael Luby. Monte-Carlo algorithms for the planar multiterminal network reliability problem. Journal of Complexity, 1(1):45–64, 1985. doi:10.1016/0885-064X(85)90021-4.

[Kle77] Victor Klee. Can the measure of $\bigcup_1^n [a_i, b_i]$ be computed in less than $O(n \log n)$ steps? The American Mathematical Monthly, 84(4):284–285, 1977. doi:10.1080/00029890.1977.11994336.

[KLM89] Richard M. Karp, Michael Luby, and Neal Madras. Monte-Carlo approximation algorithms for enumeration problems. Journal of Algorithms, 10(3):429–448, 1989. doi:10.1016/0196-6774(89)90038-2.

[Kün22] Marvin Künnemann. A tight (non-combinatorial) conditional lower bound for Klee's measure problem in 3D. In Proc. 63rd IEEE Annual Symposium on Foundations of Computer Science (FOCS), pages 555–566, 2022. doi:10.1109/FOCS54457.2022.00059.

[Len83] Hendrik W. Lenstra Jr. Integer programming with a fixed number of variables. Mathematics of Operations Research, 8(4):538–548, 1983. doi:10.1287/moor.8.4.538.

[MCV22] Kuldeep S. Meel, Sourav Chakraborty, and N. V. Vinodchandran. Estimation of the size of union of Delphic sets: Achieving independence from stream size. In Proc. 41st ACM Symposium on Principles of Database Systems (PODS), pages 41–52, 2022. doi:10.1145/3517804.3526222.

[Mut05] Shanmugavelayutham Muthukrishnan. Data streams: Algorithms and applications. Foundations and Trends® in Theoretical Computer Science, 1(2):117–236, 2005. doi:10.1561/0400000002.

[MVC21] Kuldeep S. Meel, N. V. Vinodchandran, and Sourav Chakraborty. Estimating the size of union of sets in streaming models. In Proc. 40th ACM Symposium on Principles of Database Systems (PODS), pages 126–137, 2021. doi:10.1145/3452021.3458333.

[Nek21] Yakov Nekrich. Dynamic planar point location in optimal time. In 53rd Annual ACM SIGACT Symposium on Theory of Computing (STOC), pages 1003–1014, 2021. doi:10.1145/3406325.3451100.

[NVG+24] Mridul Nandi, N. V. Vinodchandran, Arijit Ghosh, Kuldeep S. Meel, Soumit Pal, and Sourav Chakraborty. Improved streaming algorithm for the Klee's measure problem and generalizations. In Proc. 27th International Workshop on Approximation, Randomization, and Combinatorial Optimization (APPROX/RANDOM), pages 26:1–26:21, 2024. doi:10.4230/LIPIcs.APPROX/RANDOM.2024.26.

[OY91] Mark H. Overmars and Chee-Keng Yap. New upper bounds in Klee's measure problem. SIAM Journal on Computing, 20(6):1034–1045, 1991. doi:10.1137/0220065.

[PVBM23] A. Pavan, N. Variyam Vinodchandran, Arnab Bhattacharyya, and Kuldeep S. Meel. Model counting meets F_0 estimation. ACM Transactions on Database Systems, 48(3):1–28, 2023. doi:10.1145/3603496.

[RDS06] Christopher Ré, Nilesh Dalvi, and Dan Suciu. Efficient top-k query evaluation on probabilistic data. In Proc. 23rd IEEE International Conference on Data Engineering, pages 886–895, 2006. doi:10.1109/ICDE.2007.367934.

[RR23] Victor Reis and Thomas Rothvoss. The subspace flatness conjecture and faster integer programming. In Proc. 64th IEEE Annual Symposium on Foundations of Computer Science (FOCS), pages 974–988, 2023. doi:10.1109/FOCS57990.2023.00060.

[TW12] Srikanta Tirthapura and David Woodruff. Rectangle-efficient aggregation in spatial data streams. In Proc. 31st ACM Symposium on Principles of Database Systems (PODS), pages 283–294, 2012. doi:10.1145/2213556.2213595.

[Wel05] Emo Welzl. Smallest enclosing disks (balls and ellipsoids). In New Results and New Trends in Computer Science, pages 359–370. Springer, 2005. doi:10.1007/BFb0038202.

[WZ21] David P. Woodruff and Samson Zhou. Tight bounds for adversarially robust streams and sliding windows via difference estimators. In Proc. 62nd Annual IEEE Symposium on Foundations of Computer Science (FOCS), pages 1183–1196, 2021. doi:10.1109/FOCS52979.2021.00116.

[WZ25] Hong Wang and Joshua Zahl. Volume estimates for unions of convex sets, and the Kakeya set conjecture in three dimensions, 2025. arXiv:2502.17655.

Algorithm 2: Digest data structure for decremental union

parameters: n, ε
public variables: $\mathcal{X}, S, L$
function initialize($\mathcal{Y}$)
  let $\mathcal{X} := \mathcal{Y}$ and $L := \infty$
  refresh()
function refresh
  $v :=$ KLM($\mathcal{X}$)
  let L' be the unique integer such that $\frac{32 \log n}{\varepsilon^2} \leq v/2^{L'} < \frac{64 \log n}{\varepsilon^2}$
  $L := \min\{L - 1, L'\}$
  $S :=$ erase-resample($\mathcal{X}, L$)
  foreach $x \in S$ **do**
    compute and store $m_x := \#\{X \in \mathcal{X} : X.\text{contains}(x)\}$
function delete(X)
  $\mathcal{X} := \mathcal{X} \setminus X$
  foreach $x \in S$ **do**
    if X.contains(x) **then**
      $m_x := m_x - 1$
    if $m_x = 0$ **then**
      $S := S \setminus \{x\}$ and forget m_x
  if $|S| < \frac{24 \log n}{\varepsilon^2}$ **then**
    refresh()
    return true
  else
    return false

Algorithm 3: Dynamic union volume estimation with arbitrary insertions and deletions

parameters: $n, \hat{\varepsilon}$
function initialize
  for $j = 0, \ldots, \log n$ **do**
    $\mathcal{X}_j$.initialize($\emptyset$) with parameters n and $\varepsilon := \hat{\varepsilon}/(1+\log n)$
function insert(X)
  let h be the minimal integer such that $1 + \sum_{j=0}^{h} |\mathcal{X}_j| \leq 2^h$
  $\mathcal{X}_h$.initialize($\{X\} \cup \mathcal{X}_0 \cup \cdots \cup \mathcal{X}_h$) `/* merge */`
  $\mathcal{X}_j$.initialize($\emptyset$) for $j = 0, \ldots h-1$
  foreach $x \in \mathcal{X}_h.S$ **do**
    $n_x := 0$
  foreach $x \in \bigcup_{j=h+1}^{\log n} \mathcal{X}_j.S$ **do**
    if X.contains(x) **then**
      $n_x := n_x + 1$
function delete(X)
  find the bin $\mathcal{X}_j$ containing X
  refreshed := $\mathcal{X}_j$.delete(X)
  if refreshed **then**
    foreach $x \in \mathcal{X}_j.S$ **do**
      $n_x := \#\{X \in \mathcal{X}_0 \cup \cdots \cup \mathcal{X}_{j-1} : X.\text{contains}(x)\}$
  foreach $x \in \bigcup_{k=j}^{\log n} \mathcal{X}_k.S$ **do**
    if X.contains(x) **then**
      $n_x := n_x - 1$
function estimate()
  denote $S_j := \mathcal{X}_j.S$ and $L_j := \mathcal{X}_j.L$ for $j = 0, 1, \ldots, \log n$
  compute $R_j := \{x \in S_j : n_x = 0\}$ for $j = 0, 1, \ldots, \log n$
  return $\sum_{j=0}^{\log n} 2^{L_j} |R_j|$

Algorithm 4: Streaming algorithm for union volume estimation

parameters: n, ε
function initialize()
  $S^{(\ell)} := \emptyset$ and $s^{(\ell)} := 0$ for all $0 \le \ell \le 5\log(3n/\varepsilon)$
  $t := 0$
  $C := 100\log(n)/\varepsilon^2$
function insert(X)
  $t := t + 1$
  for $\ell = 0, \ldots, 5\log(3n/\varepsilon)$ **do**
    foreach $x \in S^{(\ell)}$ **do**
      if X.contains(x) **then**
        remove x from $S^{(\ell)}$
    sample $N \sim \text{Pois}(X.\text{size}/2^\ell)$
    if $N \le C$ **then**
      binary search for the smallest a with
       $N + |\{x \in S^{(\ell)} : \text{time}(x) \ge a\}| \le C$
      remove all $x \in S^{(\ell)}$ with $\text{time}(x) < a$
      $s^{(\ell)} := a$
      for $i = 1, \ldots N$ **do**
        $x := X$.sample()
        set $\text{time}(x) := t$ and add x to $S^{(\ell)}$
    else
      $s^{(\ell)} := t + 1$, $S^{(\ell)} := \emptyset$
function estimate(s)
  $L := \min\left\{\ell : s^{(\ell)} \le s\right\}$
  $R := \left\{x \in S^{(L)} : \text{time}(x) \ge s\right\}$
  return $2^L|R|$

Algorithm 5: Listing all solutions in an ILP

function list(P, k)
  if $k = 0$ **then**
    output the unique point in P
  else
    $a := \min\left\{x_k : x \in P \cap \mathbb{Z}^k\right\}$
    while $a < \infty$ **do**
      list($P[x_k \mapsto a], k-1$)
      $a := \min\left\{x_k : x \in P \cap \mathbb{Z}^k,\ x_k \geq a+1\right\}$

Algorithm 6: Union volume estimation for dynamic convex bodies

parameters: n, ε, Δ
function initialize()
  fix a prime $p \in [\Delta^d, 2\Delta^d]$
  sample $a, b \in \mathbb{F}_p$ uniformly at random
  for $\ell = 0, \ldots, \lfloor \log p \rfloor$ **do**
    instantiate a data structure v^ℓ in Theorem 7.8 with $k := \frac{100}{\varepsilon^2}$ and $\delta := \frac{1}{20 \log p}$
    $q_\ell := \lceil p/2^\ell \rceil / p$
function insert(X)
  for $\ell = 0, \ldots, \lfloor \log p \rfloor$ **do**
    if X.size $\leq 1600/(\varepsilon^2 q_\ell)$ **then**
      foreach $x \in$ filter(X, a, b, ℓ) **do**
        v^ℓ.update($x, 1$)
function delete(X)
  for $\ell = 0, \ldots, \lfloor \log p \rfloor$ **do**
    if X.size $\leq 1600/(\varepsilon^2 q_\ell)$ **then**
      foreach $x \in$ filter(X, a, b, ℓ) **do**
        v^ℓ.update($x, -1$)
function estimate()
  $L := 1 + \max\left\{\ell : v^\ell.\text{support}() = \infty\right\}$
  return $v^L.\text{support}()/q_L$